

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



Mai Phong Đăng

**TĂNG TỐC KHÁM PHÁ LỜI GIẢI TẠI THỜI
ĐIỂM SUY LUẬN TRONG BÀI TOÁN SINH MÃ
PHỨC TẠP BẰNG PHƯƠNG PHÁP TỰ CHỨNG
CẤT DỰA TRÊN PHẢN HỒI THỰC THI**

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC

Thành phố Hồ Chí Minh, tháng 7 năm 2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM

Khoa Toán – Tin học

Ngành Khoa học Dữ liệu

Mai Phong Đăng

22280008

TĂNG TỐC KHÁM PHÁ LỜI GIẢI TẠI THỜI
ĐIỂM SUY LUẬN TRONG BÀI TOÁN SINH MÃ
PHỨC TẠP BẰNG PHƯƠNG PHÁP TỰ CHỨNG
CẤT DỰA TRÊN PHẢN HỒI THỰC THI

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC

Giảng viên hướng dẫn: TS. Ngô Minh Mẫn

Thành phố Hồ Chí Minh, tháng 7 năm 2026

Tóm tắt

Test-time training cho một mô hình ngôn ngữ khả năng tự cải thiện ngay tại thời điểm suy luận, trên một bài toán khó duy nhất. Nhưng self-distillation theo kiểu on-policy rất dễ giậm chân tại chỗ: nếu mô hình không thể tự sinh ra một lời giải đúng, thì cũng chẳng có gì đúng để nó học theo. Khóa luận này tìm cách tăng tốc test-time discovery trong sinh mã phức tạp bằng *execution-guided self-distillation* — dùng execution feedback (test case thất bại, lỗi runtime) làm privileged context dẫn dắt từng bước cập nhật. Tôi đề xuất một cách tổ chức *teacher-first*: thay vì distill chính rollout thất bại của student, self-teacher sinh trước các quỹ đạo dưới feedback; một verifier và một judge lọc chúng theo tính đúng và tính độc lập; rồi student mới được distill về phía những quỹ đạo đã qua lọc đó. Cách tổ chức này đặt phương pháp đề xuất ở vị trí giao thoa giữa SDPO on-policy và SFT off-policy. Trong một đánh giá đối chiếu (matched evaluation) quy mô trung bình trên các bài toán LiveCodeBench khó bằng mô hình 4B, teacher-first vượt trội đáng kể so với baseline student-first (kiểm định Wilcoxon signed-rank). Đặc biệt trên những bài khó nhất, phương pháp này thoát được flat-reward trap — cạm bẫy vốn luôn khiến student-first giậm chân tại mức 0. Một so sánh cân bằng tính toán (compute-matched) cho thấy lợi thế này không phải là hệ quả ảo sinh ra từ khối lượng lấy mẫu: ở cùng một ngân sách generation, teacher-first vẫn chiến thắng và đạt được xác suất khám phá (discovery) mục tiêu với số lượng generation ít hơn đáng kể. Khi kích hoạt cơ chế suy nghĩ (thinking), tôi tiến hành đo lường epistemic verbalization tại thời điểm suy luận và phát hiện ra rằng: quá trình chưng cất (distill) từ một feedback-conditioned context làm suy giảm các uncertainty marker — một minh chứng trực tiếp cho hiện tượng suy giảm mà Kim và cộng sự đã mô tả. Cuối cùng, việc đánh giá pipeline trên tác vụ suy luận toán học (AIME 2026) đã vạch rõ một ranh giới áp dụng rất nghiêm ngặt (rigid domain boundary): khi feedback chỉ gói gọn trong một giá trị số tĩnh thay vì một quy trình thuật toán, teacher policy sẽ suy biến thành hành vi sao chép đáp án nông cạn mà không mang lại bất kỳ sự cải thiện năng lực nào, qua đó thiết lập nên một ràng buộc khắt khe giữa quy trình và giá trị (procedure-versus-value) đối với self-distillation tại thời điểm suy luận.

Từ khóa: test-time training, self-distillation, sinh mã, execution feedback, discovery@k, epistemic suppression, ranh giới miền.

Mục lục

Tóm tắt	1
Danh sách hình vẽ	4
Danh sách bảng	5
Danh sách từ viết tắt	6
1 Giới thiệu	7
1.1 Bối cảnh	7
1.2 Bài toán	7
1.3 Mục tiêu nghiên cứu	8
1.4 Đóng góp	8
1.5 Phạm vi	9
1.6 Cấu trúc khóa luận	9
2 Bối cảnh và Công trình liên quan	10
2.1 RLVR và GRPO	10
2.2 SDPO	11
2.2.1 Self-teacher và rich feedback	11
2.2.2 Hàm mất mát và credit assignment dày	11
2.2.3 Ba chế độ	12
2.2.4 Điều gì làm nên SDPO: ablation và scaling	12
2.3 Test-time training và discovery@k	13
2.4 Reprompt template và không gian thiết kế của nó	14
2.5 Epistemic verbalization và sự đè nén	14
2.6 SDFT và mối lo leak	15
2.7 Khoảng trống khóa luận này giải quyết	15
3 Phương pháp	16
3.1 Pipeline test-time SDPO	16
3.1.1 Ký hiệu và thiết lập	16
3.1.2 Vòng lặp chung	17
3.2 Student-first SDPO (baseline, on-policy)	17
3.3 Teacher-first SDPO (đề xuất)	18
3.3.1 Động lực	18
3.3.2 Rollout teacher và privileged context	19
3.3.3 Verifier và judge tạo good pool	19
3.3.4 Lái teacher bằng few-shot (good_only so với good_bad)	20
3.3.5 Bước distillation: reverse KL top-K căn theo token	21
3.4 Reprompt template (T1–T7)	21
3.5 Các miền: code (cốt lõi) và toán (pilot)	23
3.6 Metric và thiết kế so sánh	24

4	Thí nghiệm	25
4.1	Các mở rộng phương pháp cho nghiên cứu đầy đủ	25
4.2	Thiết lập	25
4.3	RO-method: teacher-first so với student-first	25
4.4	So sánh compute-matched và compute-to-correct (RO-method)	27
4.5	RO1: reprompt template	27
4.6	Độ vững (Robustness)	28
4.7	RO2: epistemic verbalization trên code (thinking ON)	28
4.8	Ranh giới miền (RO3): Đặc trưng hóa kiến trúc trên toán	28
4.9	Pilot toán	29
5	Thảo luận	30
5.1	Vì sao teacher-first thắng khi cân bằng compute	30
5.2	Ranh giới value-versus-procedure	30
5.3	Sự đè nén epistemic, và ý nghĩa của nó ở đây	31
5.4	Vị trí so với bài báo gốc	32
6	Giới hạn và Hướng phát triển	33
6.1	Tính chặt chẽ phương pháp luận và các kiểm chứng	33
6.2	Phạm vi và các giới hạn còn lại	33
7	Kết luận	35
	References	36
A	Siêu tham số	38
A.1	Code (LiveCodeBench v6, cốt lõi)	38
A.2	Toán (AIME 2026, pilot)	38
B	Reprompt template (nguyên văn)	40
C	Prompt teacher và judge (nguyên văn)	41
C.1	Prompt teacher và khối few-shot	41
C.2	Prompt LLM judge — code (groq llama-3.3-70b)	41
C.3	Prompt LLM judge — toán (glm-4.5-flash)	41
D	Quy đạo ví dụ (pilot toán)	43
D.1	Prompt teacher với reference bị leak (idx9, run fi5m0as1)	43
D.2	Chuỗi judge verdict	43
D.3	Hình thức đối, bản chất thì không (idx9 student eval, PRE so với POST)	44
D.4	Sự dịch chuyển phong cách ngược lại (idx8 student eval, PRE so với POST)	44
D.5	Khám phá thực trên code (idx12, run teacherfirst-...-idx12)	45
E	Kết quả theo từng seed	46
E.1	idx39 (abc393_d, khó, base pass ≈ 0.1), eval-16, diffli judge	46
E.2	idx12 (abc389_b, model-hard, model pass ≈ 0.12), eval-16	46
E.3	Đánh giá toàn diện quét frontier (8 bài $\times$ 6 seed)	46
E.4	RO1 template (nhánh SF, 6 seed), POST pass@16	47
E.5	Ablation judge $\times$ few-shot, mean POST pass (6 seed)	48
F	Dữ liệu nghiên cứu đầy đủ	49

Danh sách hình vẽ

2.1	Độ dày credit-assignment, GRPO so với SDPO. GRPO gán một scalar advantage cho cả chuỗi, nên mọi token nhận cùng tín hiệu ($O(1)$); SDPO điều kiện hóa teacher trên feedback và cấp một target mềm, K -chiều tại mỗi vị trí ($O(T \cdot K)$).	12
3.1	Hai nhánh của test-time SDPO. Student-first và teacher-first chia sẻ toàn bộ vòng lặp và chỉ tách nhau ở việc quỹ đạo nào được distill (khối tô đậm).	17
3.2	Bước teacher-first. Self-teacher sinh dưới privileged context; một verifier và judge lọc các quỹ đạo vào một good pool; các exemplar good (và tùy chọn bad) lái rollout teacher kế tiếp theo kiểu in-context; student chỉ được distill về phía các quỹ đạo good đã lọc.	19
3.3	Không gian thiết kế reprompt-template. Mỗi template thay đổi một chiều so với anchor T2: lượng thông tin (T1, T3), instruction framing (T4, T5, T6), hoặc độ sâu bộ nhớ (T7).	22
4.1	Kết quả chính (RO-method). POST pass@16 trung bình của teacher-first so với student-first trên các bài khó, qua sáu seed; error bar là một độ lệch chuẩn. Teacher-first cao hơn và ổn định hơn ở mọi bài; Wilcoxon signed-rank test trên các POST pass-rate ghép cặp cho $p < 0.01$	26
4.2	Đường cong discovery escape-zero. Pass-rate trung bình trên các bài escape-zero qua mười lăm bước test-time. Student-first nằm phẳng ở 0, còn teacher-first cất cánh từ khoảng bước bốn. Tương phản trực quan giữa một đường phẳng và một đường đi lên là dấu hiệu đặc trưng của việc thoát flat-reward-trap.	26
4.3	Compute-to-correct frontier (RO-method). Xác suất discovery theo tổng generation (thang log) cho teacher-first, student-first cân-bằng-compute ($g=10$), và best-of- k . Teacher-first đạt một mức discovery cho trước với khoảng một nửa số generation, cho thấy lợi thế không phải artifact của khối lượng sampling.	27
4.4	Sự đè nén epistemic (RO2). Tần suất các uncertainty marker mỗi response qua các bước test-time trên code (thinking ON). Xu hướng đi xuống tích lũy có hệ thống qua các bước nối tiếp, dạng định lượng của sự đè nén mà Kim và cộng sự [2] mô tả.	29
5.1	Ranh giới value-versus-procedure. Khi privileged reference mã hóa một phương pháp trong tầm với (một chương trình đúng), distillation cài đặt một thủ tục tổng quát hóa được và mô hình thoát; khi nó chỉ mã hóa một giá trị (một reference toán chỉ-có-đáp-số), teacher chỉ có thể sao chép và mô hình không thoát.	31

Danh sách bảng

2.1	Độ dày credit-assignment: target mỗi token của GRPO so với SDPO. . . .	11
3.1	Các hiện thực judge: diffliB so với LLM (cơ chế và chi phí).	20
3.2	Lái teacher bằng few-shot: exemplar good_only so với good_bad.	20
3.3	Phân loại reprompt-template: T1–T7 trên ba chiều thiết kế.	22
3.4	So sánh miền: reference và context của code (LCBv6) so với toán (AIME). . . .	23
4.1	So sánh compute-matched trên các anchor escape-zero (student-first ở $g=10$ so với teacher-first).	27
4.2	Ranh giới value-versus-procedure: kết quả escape-zero theo miền và loại feedback.	29
A.1	Siêu tham số — code (LiveCodeBench v6, cốt lõi).	38
A.2	Siêu tham số — toán (AIME 2026, pilot).	38
D.1	Chuỗi judge verdict cho idx9 (mọi quỹ đạo về danh nghĩa đều đúng). . . .	43
D.2	Chuỗi judge verdict cho idx8 (mọi quỹ đạo đều bị chấm là copy).	44
E.1	Đánh giá theo từng bước cho idx39 (eval-16, diffliB judge).	46
E.2	Đánh giá theo từng bước cho idx12 (model-hard, eval-16).	46
E.3	Tóm tắt quét frontier: 8 bài $\times$ 6 seed (SF/TF mean, win/tie/loss).	46
E.4	POST pass@16 theo từng seed cho idx64, idx77, idx58, idx78.	47
E.5	POST pass@16 theo từng seed cho idx39, idx46, idx59, idx69.	47
E.6	So sánh reprompt-template RO1 (nhánh SF, 6 seed), POST pass@16. . . .	48
E.7	Ablation judge $\times$ few-shot, mean POST pass (6 seed).	48
F.1	Kết quả chính (cơ sở cho Hình 4.1): TF/SF mean POST pass@16 theo từng bài qua sáu seed, kèm SD.	49
F.2	Đường cong discovery escape-zero (cơ sở cho Hình 4.2): mean pass-rate mỗi bước TTT.	49
F.3	Compute-to-correct frontier (cơ sở cho Hình 4.3): xác suất discovery theo tổng generation.	49
F.4	Epistemic marker (cơ sở cho Hình 4.4): uncertainty marker mỗi response mỗi bước TTT (code, thinking ON).	49

Danh sách từ viết tắt

AIME	American Invitational Mathematics Examination
EMA	Exponential Moving Average
GRPO	Group Relative Policy Optimization
JSD	Jensen–Shannon Divergence
KL	Kullback–Leibler (divergence)
LCB	LiveCodeBench
LLM	Large Language Model (mô hình ngôn ngữ lớn)
LoRA	Low-Rank Adaptation
PPO	Proximal Policy Optimization
RLRF	Reinforcement Learning with Rich Feedback
RLVR	Reinforcement Learning with Verifiable Rewards
RO	Research Objective (mục tiêu nghiên cứu)
SDFT	Self-Distillation Fine-Tuning
SDPO	Self-Distillation Policy Optimization
SF	Student-First
SFT	Supervised Fine-Tuning
TF	Teacher-First
TRL	Transformer Reinforcement Learning (thư viện)
TTT	Test-Time Training

Chương 1

Giới thiệu

Khóa luận này tìm cách tăng tốc test-time discovery trong sinh mã phức tạp bằng execution-guided self-distillation. Câu hỏi đặt ra rất thực tế: khi một mô hình ngôn ngữ chỉ được giao một bài lập trình khó duy nhất và một ngân sách nhỏ để tự cải thiện ngay tại thời điểm suy luận, cách nào biến execution feedback thành một lời giải tốt hơn hiệu quả nhất, và cách đó phát huy tác dụng trong điều kiện nào?

1.1 Bối cảnh

Reinforcement learning with verifiable rewards (RLVR) — với GRPO [8] là một đại diện phổ biến — gặp phải hạn chế về vấn đề phân bổ tín hiệu học thưa thớt (sparse credit assignment): một scalar outcome reward chỉ cho biết liệu một attempt có vượt qua hay không, do đó khi mọi rollout trong một batch đều thất bại, advantage sẽ triệt tiêu về 0 và quá trình học bị đình trệ. Đây chính xác là rào cản trên các bài toán khó mà khóa luận này hướng tới. Self-Distillation Policy Optimization (SDPO) [1] giải quyết vấn đề này bằng cách trích xuất một tín hiệu học dày đặc hơn từ rich feedback vốn đã có sẵn trong nhiều môi trường verifiable: một mô hình được điều kiện hóa trên feedback sẽ đóng vai trò self-teacher, sau đó tri thức này được distill ngược trở lại vào student. Nhờ vậy, tại thời điểm suy luận, SDPO có thể lặp đi lặp lại quá trình cải thiện trên một bài toán khó ngay cả trước khi tìm ra lời giải đúng đầu tiên [1, §5].

Hai điểm chưa trọn vẹn của bài báo gốc là động lực cho công trình này. Một, SDPO nguyên bản vẫn distill chính rollout thất bại của student — mà trên một bài khó thì có rất ít thứ đúng để học. Hai, Kim và cộng sự [2] chỉ ra rằng distill từ một context giàu thông tin có thể phản tác dụng: nó làm suy giảm mô hình bằng cách đè nén epistemic verbalization của chính nó.

1.2 Bài toán

Nghiên cứu này được đặt trong bối cảnh test-time training: tập trung vào một bài toán khó duy nhất, một verifiable reward (phần thưởng kiểm chứng được), và một số lượng giới hạn các bước tự cải thiện trước khi mô hình được đặt lại (reset). Có hai lựa chọn thiết kế quyết định tốc độ khám phá (discovery) lời giải. Thứ nhất là cách tổ chức execution feedback thành tín hiệu học — cụ thể là học từ chính attempt thất bại của student, hay học từ một attempt đúng được sinh ra dưới sự dẫn dắt của feedback và đã qua thanh lọc để đảm bảo tính độc lập. Thứ hai là cách định dạng (formulate) execution feedback trong reprompt mà self-teacher tiếp nhận [1, §7]. Vì execution feedback đóng vai trò là privileged context trong thiết lập này, cách tiếp cận trên được gọi là *execution-guided* (được dẫn dắt bởi thực thi). Khóa luận sẽ nghiên cứu cả hai lựa chọn thiết kế này nhằm mục tiêu chung là tăng tốc quá trình khám phá lời giải.

1.3 Mục tiêu nghiên cứu

- **RO-method.** Xác định liệu cách tổ chức teacher-first có vượt trội hơn baseline student-first trong việc khám phá lời giải cho các bài toán code khó hay không, và liệu lợi thế đó có được duy trì khi cân bằng tài nguyên tính toán (compute-matched) hay không.
- **RO1.** Xác định xem cách định dạng (formulate) execution feedback thông qua reprompt-template có ảnh hưởng đến tốc độ khám phá (discovery) hay không, và nếu có thì dưới điều kiện (regime) nào.
- **RO2.** Đo lường xem liệu quá trình tự chưng cất (self-distillation) từ một ngữ cảnh giàu thông tin có làm suy giảm các biểu đạt nhận thức (epistemic verbalization) tại thời điểm suy luận trên mã nguồn hay không.
- **RO3.** Mô tả đặc trưng các giới hạn về cấu trúc và miền ứng dụng của phương pháp tự chưng cất được dẫn dắt bởi thực thi (execution-guided self-distillation), đặc biệt khi chuyển từ miền code mang tính quy trình (procedural) sang các ngữ cảnh toán học chỉ mang giá trị tĩnh (value-only).

Cả bốn mục tiêu này đều được đánh giá và giải quyết xuyên suốt khung thực nghiệm của khóa luận.

1.4 Đóng góp

1. **Đề xuất biến thể teacher-first cho phương pháp execution-guided self-distillation.** Student được chưng cất (distill) hướng theo các generation của teacher, sau khi các generation này đã được một verifier và một judge sàng lọc. Cách tổ chức này đặt phương pháp ở vị trí giao thoa giữa SDPO on-policy và SFT off-policy, đi kèm một bộ lọc đủ khắt khe để không bao giờ học nhầm (distill) từ một bản sao chép (Chương 3).
2. **Teacher-first tăng tốc đáng kể quá trình khám phá (discovery) trên code, và lợi thế này hoàn toàn công bằng về mặt tính toán (compute-fair).** Trong một thử nghiệm đối chiếu (matched evaluation, 8 bài $\times$ 6 seed), teacher-first đánh bại student-first (theo kiểm định Wilcoxon signed-rank) và thoát khỏi cạm bẫy flat-reward trap trên chính những bài toán khó nhất. Một so sánh cân bằng tính toán (compute-matched) kết hợp với đường biên compute-to-correct cho thấy lợi ích này không phải là hệ quả ảo sinh ra từ khối lượng lấy mẫu (sampling-volume artifact) (Chương 4).
3. **Làm rõ hiệu ứng của reprompt-template**, thể hiện rõ nhất trên các bài toán khó: một template mang tính kích thích suy luận (reasoning-inducing) sẽ cho hiệu năng cao hơn template mỏ neo (anchor), và template mỏ neo lại vượt trội hơn một template tối giản (§4.5).
4. **Đo lường được hiện tượng epistemic suppression (suy giảm biểu đạt nhận thức) trên tác vụ code** (khi bật thinking): quá trình distill từ một feedback-conditioned context làm suy giảm các marker thể hiện sự không chắc chắn (uncertainty markers), và mức độ giảm này tích lũy dần qua các bước học (§4.7).
5. **Đặc trưng hóa ranh giới miền ứng dụng giữa quy trình và giá trị (value-versus-procedure).** Khi pipeline rơi vào trạng thái zero-reward trap không thể thoát trên bộ dữ liệu AIME 2026, nó đã vạch rõ một giới hạn nền tảng của cách tiếp cận này: quá trình self-distillation đòi hỏi một tín hiệu học mang tính quy trình (như các bộ test case dày đặc trong code) để có thể nội hóa được một phương pháp, và sẽ bị đình trệ khi chỉ có các tham chiếu tĩnh, thuần-giá-trị (value-only reference)

để học hỏi (§4.8, Chương 5).

1.5 Phạm vi

Khóa luận sử dụng test-time training kết hợp với LoRA adapter; bài toán sinh mã phức tạp trên LiveCodeBench v6 [6] đóng vai trò là miền thực nghiệm cốt lõi, trong khi AIME 2026 được sử dụng làm môi trường toán học đối chứng. Kết quả nghiên cứu tập trung vào họ mô hình lập luận ở quy mô 4B. Về bản chất, khóa luận này mang tính chất khảo sát thực nghiệm trong giới hạn tài nguyên tính toán cá nhân (personal compute scale) — nhằm vạch rõ các ranh giới miền áp dụng và đúc kết những kiến thức nền tảng, thay vì khẳng định bất kỳ quy luật mở rộng (scaling laws) phổ quát nào.

1.6 Cấu trúc khóa luận

Chương 2 trình bày tổng quan về bối cảnh và các công trình nghiên cứu liên quan. Chương 3 đặc tả phương pháp đề xuất, bao gồm cả các tiện ích mở rộng khi kích hoạt cơ chế suy nghĩ (thinking-on). Chương 4 báo cáo các thực nghiệm sinh mã trên nhiều seed, phép so sánh cân bằng tính toán (compute-matched), kết quả về sự suy giảm biểu đạt nhận thức (epistemic suppression), và ranh giới áp dụng trên miền toán học. Chương 5 thảo luận về các cơ chế cốt lõi chi phối những kết quả này. Chương 6 trình bày các giới hạn về cấu trúc còn tồn tại cùng những hướng phát triển trong tương lai. Chương 7 khép lại khóa luận bằng phần kết luận.

Chương 2

Bối cảnh và Công trình liên quan

Chương này trình bày nền tảng bối cảnh mà khóa luận được xây dựng dựa trên đó: từ thiết lập học tăng cường (reinforcement learning) làm động lực cho self-distillation ra đời (§2.1), qua phương pháp SDPO và hàm mất mát của nó (§2.2), đến chế độ test-time cùng hệ đo lường (metric) tương ứng (§2.3), không gian thiết kế reprompt-template (§2.4), những cảnh báo về hiện tượng epistemic suppression (§2.5), một phương pháp self-distillation tương đồng (§2.6), và khép lại bằng khoảng trống nghiên cứu mà khóa luận này hướng đến giải quyết (§2.7).

2.1 RLVR và GRPO

Reinforcement learning with verifiable rewards (RLVR) hiện là phương pháp post-training chủ đạo cho các tác vụ lập trình (code) và toán: một bộ kiểm tra (checker) tự động cung cấp phần thưởng (reward), thay vì dựa vào human preference như trong reinforcement learning from human feedback [15] hay các bản đơn giản hóa của nó về tối ưu hóa sở thích (preference-optimization) [16]. Group Relative Policy Optimization (GRPO) [8] là một đại diện phổ biến nhất: phương pháp này lấy mẫu (sample) một nhóm các rollout cho mỗi bài toán, rồi gán cho mỗi rollout một advantage tương đối so với trung bình nhóm, $A_{i,t} = r_i - \text{mean}_j\{r_j\}$ — một hằng số được giữ nguyên cho mọi token trong một chuỗi.

Ưu điểm của GRPO là nó loại bỏ được value network riêng biệt mà các phương pháp kiểu PPO [14] vẫn cần: trung bình nhóm tự đóng vai trò là baseline, còn advantage được ước lượng trực tiếp từ độ phân tán của reward trong nhóm đã sample [8]. Nhờ vậy, phương pháp này được giữ ở mức tối giản, và đó cũng là một phần lý do nó trở thành lựa chọn mặc định cho các miền verifiable.

Tuy nhiên, chính sự tối giản đó lại là điểm yếu trong việc phân bổ tín hiệu học (credit assignment). Một scalar outcome reward chỉ cho biết liệu toàn bộ rollout có thành công hay không, chứ không ghi nhận được token nào thực sự chịu trách nhiệm — dẫn đến tín hiệu học rất thưa thớt [1]. Process supervision — chấm điểm từng bước lập luận trung gian thay vì chỉ đánh giá kết quả cuối cùng [22] — là một cách giải quyết sự thưa thớt này; trong khi đó, SDPO đi theo một hướng khác: trích xuất một target dày đặc (dense target) theo từng token ngay từ chính feedback. Điểm yếu này sẽ trở thành thất bại hoàn toàn khi mọi rollout trong một nhóm đều sai: advantage tương đối triệt tiêu về 0, không có gradient nào được truyền đi, và phương pháp không thể học trên bài toán đó cho đến khi nó tìm ra lời giải đúng ít nhất một lần. Ở những bài toán khó — nơi sự thành công vốn rất hiếm hoi — đây chính xác là rào cản khiến quá trình học bị đình trệ.

Một giải pháp khắc phục tự nhiên là distill từ một teacher mạnh hơn. Nhưng điều đó yêu cầu phải có sẵn một teacher vượt trội hơn student, trong khi mục tiêu ở đây là nâng cao giới hạn năng lực (capability ceiling) thay vì nén một mức năng lực đã biết [1]. Nghịch lý này chính là động lực để tìm kiếm một teacher không đến từ bên ngoài, mà được dẫn xuất từ chính bản thân student.

2.2 SDPO

2.2.1 Self-teacher và rich feedback

Hübottner và cộng sự [1] nhận ra rằng nhiều môi trường verifiable vốn đã cung cấp sẵn tokenized feedback — lỗi thời gian chạy (runtime errors), diff của test case thất bại, hay nhận xét của judge — nhưng những tín hiệu này lại bị lãng phí khi mọi thông tin bị thu gọn thành một scalar reward duy nhất. Họ đề xuất reinforcement learning with rich feedback (RLRF) như một khuôn khổ (framework) nhằm bảo toàn tín hiệu đó, và SDPO chính là một cách hiện thực hóa của khuôn khổ này. Cơ chế cốt lõi biến ý tưởng này thành hiện thực là in-context learning: cùng một mô hình, khi được điều kiện hóa thêm trên feedback f bên cạnh câu hỏi x , thường có khả năng tự định vị được lỗi sai của chính mình. Mô hình được điều kiện hóa này, $\pi_\theta(\cdot \mid x, f)$, đóng vai trò là *self-teacher*; trong khi mô hình không điều kiện $\pi_\theta(\cdot \mid x)$ đóng vai trò là student. Vì cả hai cùng chia sẻ chung tập trọng số θ , đây là quá trình self-distillation đúng nghĩa, kế thừa nền tảng của knowledge distillation [13] — bao gồm cả các biến thể mức chuỗi (sequence-level) cho ngôn ngữ [24] — chỉ khác ở chỗ nguồn lợi thế của teacher đến từ feedback thay vì từ một mạng neural lớn hơn.

2.2.2 Hàm mất mát và credit assignment dày

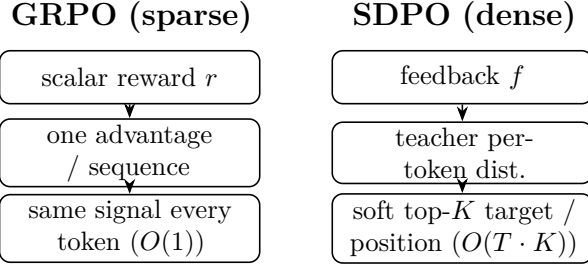
SDPO distill phân bố next-token mang tính hồi cứu (retrospective) của self-teacher vào student thông qua một hàm KL phân bố theo từng token:

$$\mathcal{L}_{\text{SDPO}}(\theta) = \sum_{t=1}^{|y|} \text{KL} \left(\pi_\theta(\cdot \mid x, y_{<t}) \parallel \text{stopgrad } \pi_\theta(\cdot \mid x, f, y_{<t}) \right),$$

trong đó `stopgrad` ngăn không cho teacher suy biến (collapse) về student và bỏ qua f [1]. Vì teacher được ngắt gradient (detached), đạo hàm tại một logit của student trở thành $\partial \mathcal{L} / \partial z_v = \pi_S(v) - \pi_T(v)$ — nghĩa là optimizer sẽ gia tăng giá trị của bất kỳ logit nào được teacher gán xác suất cao hơn so với student. So với GRPO, khác biệt lớn nhất nằm ở mật độ (density) của target:

Bảng 2.1: Độ dày credit-assignment: target mỗi token của GRPO so với SDPO.

Phương pháp	Target mỗi token	Tín hiệu mỗi chuỗi
GRPO	scalar advantage $\times$ one-hot	$O(1)$
SDPO (mức chuỗi)	scalar $\times$ soft distribution	$O(1)$
SDPO (mức token)	soft distribution, top token	$O(T)$
SDPO (mức logit)	soft distribution trên top- K vocab	$O(T \cdot K)$



Hình 2.1: Độ dày credit-assignment, GRPO so với SDPO. GRPO gán một scalar advantage cho cả chuỗi, nên mọi token nhận cùng tín hiệu ($O(1)$); SDPO điều kiện hóa teacher trên feedback và cấp một target mềm, K -chiều tại mỗi vị trí ($O(T \cdot K)$).

Ở hình thái đầy đủ nhất — mức logit — target là một phân bố K -chiều tại mọi vị trí, thay vì một scalar duy nhất cho toàn bộ chuỗi; đây chính là nguồn gốc tạo nên hiệu suất lấy mẫu (sample efficiency) vượt trội mà SDPO đạt được [1]. Để giới hạn chi phí tính toán trên một vocabulary khoảng 150k, phương pháp này chỉ giữ lại top- K logit của teacher ($K=100$, hoặc 20 đối với cấu hình code [4]) cộng thêm một nhóm đuôi (tail bucket). Chiều (direction) của KL được tham số hóa qua α : forward KL ($\alpha=0$) có tính chất mode-covering, reverse KL ($\alpha=1$) có tính chất mode-seeking, còn JSD ($\alpha=0.5$) đóng vai trò nội suy giữa hai cực này — lựa chọn này tuân theo Agarwal và cộng sự [11] để đảm bảo tính ổn định. Self-teacher cũng cần được điều chuẩn (regularize), thường bằng một EMA của trọng số student hoặc một phép nội suy trust-region, vì nếu thiếu bước này, teacher sẽ phân kỳ (diverge) [1]. Cách hiện thực (implementation) dùng trong khóa luận này được đặc tả kỹ hơn ở §3.3.5; toàn bộ phần dẫn xuất toán học được đính kèm trong ghi chú dự án.

2.2.3 Ba chế độ

Bài báo gốc kiểm chứng SDPO trên ba thiết lập khác nhau. Ở thiết lập RLVR thuần, khi không có sẵn rich feedback (native), họ đã giả lập feedback bằng cách lấy một rollout thành công trong batch làm tham chiếu (reference) cho một rollout thất bại — và SDPO vẫn vượt trội đáng kể so với GRPO, đạt được độ chính xác mà GRPO cần 5 giờ chỉ trong 50 phút trên một tác vụ hóa học [1]. Ở thiết lập RLRF với execution feedback thực trên bộ LiveCodeBench v6 [6], SDPO đạt 48.8% so với 41.2% của GRPO, đồng thời bắt kịp độ chính xác cuối cùng của GRPO với tổng số generation ít hơn khoảng 4 lần. Thiết lập thứ ba — test-time discovery trên một bài toán khó duy nhất — chính là phạm vi nghiên cứu của khóa luận này, và sẽ được thảo luận ngay sau đây.

2.2.4 Điều gì làm nên SDPO: ablation và scaling

Ba phát hiện từ phần phân tích của bài báo gốc có ảnh hưởng trực tiếp đến các lựa chọn thiết kế trong khóa luận này.

Thứ nhất, cả hai thành phần của SDPO đều thực sự cần thiết. Khi phân rã phương pháp thành các biến thể mức logit, mức token và mức chuỗi, thứ tự hiệu năng logit > token > chuỗi > GRPO [1, §4.2] cho thấy cả rich feedback và credit assignment dày đều cùng đóng góp vào sự cải thiện, thay vì chỉ một yếu tố đóng vai trò quyết định. Một phân tích cắt bỏ (ablation) theo loại feedback (Bảng 6 của họ) cũng chỉ ra kết luận tương tự: chỉ dùng output môi trường đạt 39.9% training accuracy, chỉ dùng lời giải trước đó của mô hình đạt 42.6%, nhưng kết hợp cả hai lại nâng độ chính xác lên tới 48.3% — nghĩa là hai

nguồn tín hiệu này mang tính hỗ trợ cho nhau. Ngược lại, nếu đưa attempt thất bại của student vào context của teacher, độ chính xác tụt xuống 44.5% và entropy giảm 0.23, vì lúc này teacher bị thiên lệch về phía student và mất đi tính đa dạng. Kết quả này chính là một phần động lực cho cách tổ chức teacher-first ở §3.3, vốn chỉ distill các generation của teacher đã qua thanh lọc, thay vì tái sử dụng attempt thất bại của student.

Thứ hai, khả năng tự dạy (self-teaching) là một năng lực đột phá xuất hiện theo quy mô (emergent with scale) [1, §4.1]. Trên Qwen3-8B, SDPO vượt xa GRPO; trên Qwen3-0.6B hai phương pháp cho kết quả tương đương; còn trên Qwen2.5-1.5B, SDPO lại kém hơn GRPO. Lý do là phương pháp này đòi hỏi năng lực in-context learning đủ mạnh để mô hình được điều kiện hóa thực sự đóng vai trò là một teacher hữu ích. Đặc điểm này đặt mô hình 4B dùng trong khóa luận vào đúng phân khúc mà SDPO được kỳ vọng sẽ phát huy tác dụng, đồng thời gợi ý rằng trên một mô hình mạnh hơn, baseline student-first sẽ tự cải thiện và thu hẹp khoảng cách với teacher-first — một dự đoán sẽ được thảo luận lại ở Chương 6.

Thứ ba, self-teacher được tự mỗi (bootstrap) liên tục chứ không cố định ngay từ đầu. Nó được cập nhật đồng thời với student, độ chính xác của nó tăng dần trong quá trình huấn luyện, và cuối cùng student còn vượt qua chính teacher ban đầu — do đó, tiềm năng của phương pháp không bị giới hạn trần bởi teacher khởi điểm [1, §4.3]. Regularization chính là yếu tố giữ cho quá trình này ổn định: một teacher dùng trust-region (50.6%) nhỉnh hơn một chút so với teacher dùng EMA (49.3%) và teacher đóng băng (48.8%), trong khi một teacher không có regularization sẽ hoàn toàn phân kỳ. SDPO cũng ít bị hiện tượng quên (forgetting) hơn so với việc supervised fine-tuning trực tiếp trên output của self-teacher, xét trên các benchmark held-out — mặc dù có quên nhiều hơn GRPO một chút — cho thấy bản chất on-policy của bước cập nhật giúp bảo toàn năng lực nền tảng tốt hơn [1, §4.4]. Một biến thể hybrid kết hợp lợi thế của GRPO và SDPO tỏ ra hữu ích cho các mô hình yếu, nhưng không mang lại lợi ích đáng kể cho các mô hình mạnh [1, §4.5].

2.3 Test-time training và discovery@k

Trong thiết lập test-time [1, §5], một bài toán khó duy nhất được cố định, phần thưởng (reward) có dạng nhị phân, và mô hình phải khám phá ra lời giải càng nhanh càng tốt. Thay vì nối một lịch sử ngày càng dài vào context như cách tiếp cận multi-turn reprompting vẫn làm, SDPO nén thẳng feedback vào trọng số bằng cách distill $\pi_\theta(\cdot \mid x, c)$ vào $\pi_\theta(\cdot \mid x)$ — qua đó tránh được giới hạn của context-window.

Đại lượng đo lường (metric) được sử dụng ở đây là discovery@k [1, Def. 5.1]: xác suất giải được bài toán trong k attempt đầu tiên, $P(T \leq k)$ với thời gian khám phá $T = \min\{k : r(y_k) = 1\}$. Đại lượng này tổng quát hóa pass@k [9] từ việc lấy mẫu độc lập cùng phân bố (i.i.d.) sang các thuật toán tuần tự có chia sẻ trạng thái hoặc cập nhật trọng số giữa các attempt, và thu về đúng pass@k khi thuật toán chỉ là best-of-k thuần túy. Nó chia sẻ chung nền tảng toán học với hiệu năng time-to-termination của Dolan và Moré [12]. Về mặt thực nghiệm, SDPO khám phá được lời giải cho 53.2% các bài toán cực khó, so với 41.5% của best-of-k, và trên một số bài toán, đây là phương pháp duy nhất có thể giải được — mặc dù độ chính xác của self-teacher ban đầu vẫn dưới 1% trên gần như mọi bài khó [1]. Đây chính là điểm cốt lõi nhất: credit assignment dày đặc cho phép mô hình cải thiện dần ngay cả trước khi nó sinh ra được lời giải đúng đầu tiên.

Điều này định vị công trình vào một bức tranh nghiên cứu rộng lớn hơn về inference-time compute. Việc lấy mẫu lặp (repeated sampling) giúp tăng độ phủ (coverage) trên các

bài toán khó, mở rộng theo luật lũy thừa (power law) của ngân sách lấy mẫu [20]; self-consistency tổng hợp nhiều reasoning path đã lấy mẫu thành một đáp án đáng tin cậy hơn [18]; và test-time scaling có kiểm soát ngân sách sẽ cải thiện khả năng lập luận bằng cách tiêu tốn bổ sung token cho mỗi bài [21]. Tương đồng nhất về mặt triết lý là phương pháp self-taught reasoning [19], tự mỗi (bootstrap) một mô hình từ chính các rationale do nó tự sinh ra; phương pháp teacher-first cũng học theo hướng tương tự — từ các quỹ đạo tự sinh đã qua thanh lọc — nhưng distill một target dày đặc theo từng token thay vì fine-tune trực tiếp trên văn bản.

2.4 Reprompt template và không gian thiết kế của nó

Hành vi của self-teacher bị chi phối bởi template, thứ quyết định cách x và f được định dạng để đưa vào context của nó. Bài báo gốc sử dụng một template dạng slot (Bảng 2 của họ), chèn một rollout thành công trước đó, feedback từ môi trường, và một chỉ dẫn kết thúc; họ ghi nhận rằng hiệu năng ”không nhảy với các biến thể cú pháp” của template, trong khi loại feedback lại có ảnh hưởng rõ rệt hơn — một phân tích cắt bỏ (ablation) (Bảng 6 của họ) cho thấy output của môi trường và lời giải của chính mô hình mang tính hỗ trợ cho nhau, trong khi việc đưa attempt thất bại của student vào context của teacher lại làm suy giảm tính đa dạng [1].

Từ đó mở ra hai khoảng trống nghiên cứu. Bài báo trên chỉ kiểm chứng biến thể cú pháp chứ chưa khảo sát biến thể ngữ nghĩa hay chỉ dẫn, và chính các tác giả cũng nhấn mạnh rằng việc nghiên cứu hệ thống cách reprompt template ảnh hưởng đến hành vi vẫn là hướng mở (future work) [1, §7]. Khóa luận này tổ chức không gian thiết kế template theo ba chiều: lượng thông tin — thúc đẩy bởi phát hiện của Kim và cộng sự rằng độ giàu của context chi phối mức độ đè nén [2]; cách đóng khung chỉ dẫn (instruction framing) — trực tiếp giải quyết câu hỏi mở vừa nêu; và chiều sâu bộ nhớ (memory depth) — thúc đẩy bởi tính tích lũy tuần tự của thiết lập test-time. Bấy template ở §3.4 lấy mẫu đại diện cho các điểm trên ba chiều này, với template mặc định của bài báo gốc đóng vai trò làm điểm neo (anchor).

2.5 Epistemic verbalization và sự đè nén

Kim và cộng sự [2] là mặt cảnh báo đối trọng với SDPO. Họ chỉ ra rằng distill từ một context giàu thông tin có thể phản tác dụng: nó làm suy giảm mô hình bằng cách đè nén *epistemic verbalization* của chính nó — các uncertainty marker như ”wait” hay ”maybe” thường đi kèm chain-of-thought reasoning [17]. Phân tích của họ gắn độ lớn của sự đè nén này với mutual information $I(y; c | x)$ giữa output và context: context càng giàu thông tin, sự đè nén càng mạnh. Họ khảo sát dài này bằng cách thay đổi nội dung context của teacher, từ rỗng ($c = \emptyset$), qua một lời giải đã bóc phần thinking kèm một đáp án sinh lại, cho tới một lời giải đầy đủ — và thấy sự đè nén tăng dần theo lượng thông tin context mang. Hiệu ứng này còn tích lũy qua các bước distillation nối tiếp nhau, nên họ khuyến nghị đóng băng teacher (không cập nhật EMA) để phá vòng lặp feedback — một khuyến nghị đi ngược lại ưu tiên của bài báo gốc, vốn muốn một teacher chuyển động và được regularize (§2.2.4). Nhìn rộng hơn, các mô hình ngôn ngữ vốn đã được hiệu chỉnh một phần về những gì chúng biết [23], nên việc đè nén các marker biểu thị bất định này là một con đường khá hợp lý dẫn tới suy giảm năng lực.

Với khóa luận này, sự liên quan nằm ở hai điểm. Một, rủi ro đè nén sắc bén nhất đúng vào lúc context chứa sẵn đáp án — chính là math leak regime được nghiên cứu trong pilot (§4.9), nơi teacher hoàn toàn có thể chép đáp án thay vì suy dẫn ra nó. Hai, sự tích lũy qua các bước lại đúng là điều mà thiết lập test-time áp dụng lặp đi lặp lại, vì mỗi bước TTT đều distill từ cùng một answer-bearing context.

2.6 SDFT và mối lo leak

Shenfeld và cộng sự [3] đề xuất self-distillation fine-tuning (SDFT) cho continual learning — một kiểu distillation on-policy, thay-đổi-tối-thiểu, hướng về phía generation được điều kiện hóa của chính mô hình. Đây là một phương pháp chị em với SDPO: cũng dùng mô hình làm teacher của chính nó, nhưng mục đích lại là bảo toàn năng lực cũ trong khi thích nghi với cái mới, nên việc tối thiểu hóa các thay đổi ngoài ý muốn trở thành một mục tiêu thiết kế rõ ràng. Tổ chức teacher-first của khóa luận này gần với SDFT ở chỗ nó distill về phía các quỹ đạo mô-hình-tự-sinh đã lọc, nhưng dùng execution feedback làm context điều kiện thay vì một demonstration cố định (§3.3.1). Khung thay-đổi-tối-thiểu của SDFT cũng làm sắc nét thêm câu hỏi về leak: nếu distillation truyền đi nhiều hơn nội dung dự kiến, thì thứ thực sự đang bị chép là gì — đây chính là câu hỏi mà math pilot trả lời một cách rất cụ thể.

2.7 Khoảng trống khóa luận này giải quyết

Bài báo gốc nghiên cứu một SDPO student-first, on-policy, và dồn phần lớn phân tích vào các chế độ train-time; nó có chạm tới thiết lập test-time nhưng chưa nghiên cứu reprompt template ở đó, cũng chưa đo hành vi epistemic. Kim và cộng sự thì đặc trưng hóa epistemic suppression, nhưng không trong thiết lập test-time code-discovery. Còn Shenfeld và cộng sự tối thiểu hóa thay đổi cho continual learning, chứ không nhắm tới việc tối đa hóa discovery trên một bài khó duy nhất. Khoảng trống, vì vậy, có ba phần. Thứ nhất là một tổ chức teacher-first của execution-guided self-distillation, distill các generation teacher đã lọc, nằm giữa SDPO và SFT — cùng câu hỏi liệu nó có thực sự tăng tốc test-time discovery hay không (RO-method). Thứ hai là ảnh hưởng của cách formulate reprompt-template lên discovery đó, đúng câu hỏi mở mà bài báo gốc từng nêu tên (RO1). Thứ ba là một đặc trưng hóa cho biết khi nào cách tiếp cận này giúp được, đóng khung dưới dạng ranh giới value-versus-procedure mà tương phản code-versus-math phơi bày ra. Phần còn lại của khóa luận sẽ lần lượt giải quyết từng phần trong ba khoảng trống này.

Chương 3

Phương pháp

Chương này đặc tả phương pháp đủ chi tiết để có thể tái lập. Đối tượng nghiên cứu là test-time SDPO [1, §5]: một bài toán khó duy nhất được cố định, mô hình tự cải thiện qua một số nhỏ bước huấn luyện ngay tại thời điểm suy luận (test-time training, TTT), và ta đo xem nó khám phá được lời giải tốt đến đâu. Vì privileged context điều khiển bước cập nhật chính là execution feedback (test thất bại, lỗi thời gian chạy), ta gọi chế độ này là execution-guided self-distillation, và mục tiêu là tăng tốc quá trình test-time discovery đó trong sinh mã phức tạp. Trong chế độ này, khóa luận đối chiếu hai cách tổ chức bước cập nhật — student-first (baseline của bài báo gốc [1]) và teacher-first (một biến thể do người hướng dẫn đề xuất) — rồi nghiên cứu tiếp cách reprompt template formulate execution feedback (RO1).

Phạm vi miền được thiết kế bất đối xứng có chủ đích. Sinh mã là cốt lõi (LiveCodeBench v6 [6]); toán chỉ đóng vai một pilot và một điểm tương phản (AIME 2026 [10]), dùng để đặc trưng hóa ranh giới của cơ chế chứ không phải để chứng minh nó hoạt động ở mọi nơi. Sự phân chia đó được giữ nhất quán xuyên suốt: mọi thiết lập toán đều mang theo các caveat tương ứng về mô hình và reference (§3.5, Chương 6).

Ký hiệu trong chương này phản chiếu bài báo gốc, Hübötter và cộng sự [1], để hai bên có thể đọc song song với nhau.

3.1 Pipeline test-time SDPO

3.1.1 Ký hiệu và thiết lập

Cố định một bài toán x (đề bài). Hai policy chia sẻ cùng một bộ trọng số θ :

- student $\pi_\theta(\cdot \mid x)$, chỉ thấy đề bài, và
- self-teacher $\pi_\theta(\cdot \mid x, c)$, cùng mô hình đó nhưng được điều kiện hóa thêm trên một privileged context c .

Trong SDPO, c đóng đúng vai "feedback" f của bài báo gốc: thông tin hồi tưởng (một lỗi thời gian chạy, một test thất bại, hay một lời giải reference) giúp mô hình nhìn lại và định vị được lỗi của mình [1, §2]. Vì teacher và student chỉ khác nhau ở input context, đây là self-distillation đúng nghĩa: mô hình tự học bằng cách kéo phân bố next-token của nó (khi không có context) về phía phân bố của chính nó khi được điều kiện hóa trên c .

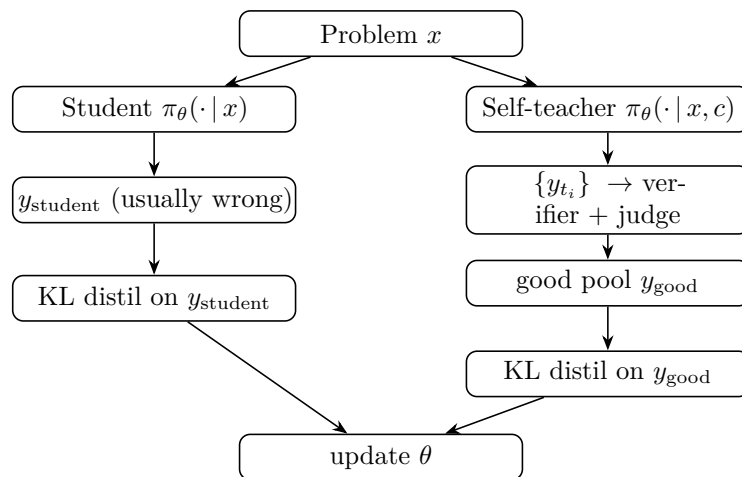
Thiết lập test-time theo đúng Hübötter và cộng sự [1, §5]. Với mỗi bài x , mô hình được reset về một base checkpoint rồi chạy K bước TTT chỉ trên bài đó. Reward là verifiable — chấm theo mức (graded) cho code, và nhị phân cho toán. Mục tiêu không phải tổng quát hóa sang bài khác, mà là khám phá lời giải cho x càng sớm càng tốt; metric dùng ở đây là discovery@k [1, Def. 5.1] (§3.6).

3.1.2 Vòng lặp chung

Cả hai nhánh chia sẻ một khung xương giống hệt nhau. Điều duy nhất khác biệt giữa chúng là *quỹ đạo nào được distill*:

```
load base model + LoRA + optimizer (AdamW, lr = 1e-5)
PRE-eval (student pi_theta(*|x) only, N_eval samples)           # baseline
for step in 1..K:
    feedback          <- build_feedback(student attempt, verifier) # rich feedback
    {trajectories}    <- generate(...)                             # arm-specific
    {distill_targets} <- select(...)                               # arm-specific (the
    ↪ core difference)
    if distill_targets != {}:
        theta <- theta - lr * grad_theta L_KL(distill_targets)    #
    ↪ one optimizer step
    log step stats -> W&B
POST-eval (student pi_theta(*|x) only, N_eval samples)         # after TTT
report PRE->POST and the discovery curve
```

Hình 3.1 phác họa hai nhánh này; chúng chỉ tách nhau đúng tại khối được tô đậm.



Hình 3.1: Hai nhánh của test-time SDPO. Student-first và teacher-first chia sẻ toàn bộ vòng lặp và chỉ tách nhau ở việc quỹ đạo nào được distill (khối tô đậm).

Cả sự khác biệt này thu gọn được trong một câu: student-first distill chính attempt thất bại của student, còn teacher-first distill một quỹ đạo đúng, đã lọc, do teacher sinh ra. Mọi thứ khác — hàm mất mát, mô hình, hyperparameter — đều được giữ y nguyên để cô lập đúng một biến số đó.

3.2 Student-first SDPO (baseline, on-policy)

Đây là thuật toán gốc của Hübottner và cộng sự [1], hiện thực qua SDPOTrainer của TRL [5] (script 07_discovery_curve.py).

Ở mỗi bước, student sample một rollout $y \sim \pi_\theta(\cdot | x)$ theo kiểu on-policy. Verifier chấm y và sinh ra feedback f . Self-teacher $\pi_\theta(\cdot | x, f)$ sau đó chấm lại *từng token* của chính y : cho f , phân bố next-token đúng lẽ ra phải là gì tại mỗi vị trí? Student được kéo về phía phân bố hồi tưởng này bằng một KL theo từng token:

$$\mathcal{L}_{\text{SDPO}}(\theta) = \sum_{t=1}^{|y|} \text{KL} \left(\pi_{\theta}(\cdot \mid x, y_{<t}) \parallel \text{stopgrad } \pi_{\theta}(\cdot \mid x, f, y_{<t}) \right).$$

`stopgrad` chặn gradient chảy qua teacher, nhờ vậy teacher không sụp vào student và bỏ qua f [1, §2]. Gradient và xấp xỉ top-K này được chia sẻ với teacher-first, sẽ được nói kỹ hơn ở §3.3.5.

Có một chế độ thất bại trên bài khó, và chính nó thúc đẩy toàn bộ phần tiếp theo. Vì rollout là on-policy, trên một bài mà bản thân student thuần (chưa được điều kiện hóa) hiếm khi giải được, gần như mọi rollout đều sai — feedback vì thế nghèo nàn, tín hiệu distillation yếu và bất ổn. Đây chính là flat-reward trap: không có rollout đúng nào để học theo cả. Hübottner và cộng sự [1, §5] cho thấy SDPO vẫn có thể lập từ rich feedback ngay cả khi độ chính xác teacher ban đầu dưới 1%, nhưng họ dùng Qwen3-8B; còn trên mô hình 4B yếu hơn dùng ở đây (§3.5), hành vi kẹt-ở-0 lại thực sự xuất hiện — và đó chính xác là thứ teacher-first nhắm tới giải quyết.

3.3 Teacher-first SDPO (đề xuất)

3.3.1 Động lực

Hai vấn đề của student-first là động lực cho biến thể này.

Vấn đề thứ nhất là flat-reward trap vừa nêu ở §3.2: khi student chưa từng tung ra một attempt đúng, thì không có gì đúng để distill cả.

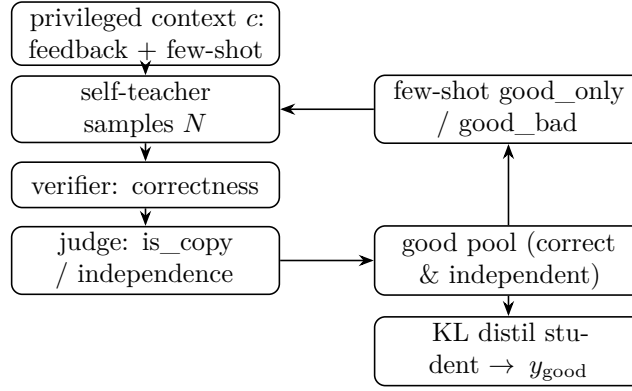
Vấn đề thứ hai là information leak [2]. Nếu c chứa một lời giải reference, teacher sẽ chấm lại theo hướng "càng gần reference càng tốt" — nên qua nhiều bước test-time, student dần hội tụ về việc *sao chép* reference thay vì thực sự học lập luận. Kim và cộng sự [2] hình thức hóa hiện tượng này: khi $I(y; c \mid x)$, tức độ giàu thông tin của context, càng cao, mô hình càng dễ nén epistemic verbalization và càng suy giảm.

Thay đổi cốt lõi ở đây là đảo ngược thứ tự. Teacher sinh trước; các quỹ đạo đúng và độc lập được lọc ra; và student được distill về phía chúng. KL được tính trên y_{good} (do teacher sinh, đã qua lọc), chứ không phải trên y_{student} (một attempt thất bại):

$$\mathcal{L}_{\text{TF}}(\theta) = \sum_{y \in \mathcal{G}} \sum_{t=1}^{|y|} \text{KL} \left(\pi_{\theta}(\cdot \mid x, y_{<t}) \parallel \text{stopgrad } \pi_{\theta}(\cdot \mid x, c, y_{<t}) \right),$$

trong đó $\mathcal{G}$ là good pool (§3.3.3). Về mặt khái niệm, teacher-first nằm ở đâu đó giữa SDPO (on-policy) và SFT (off-policy). Nó giống SDFT [3] ở chỗ distill về phía các generation do chính mô hình (được điều kiện hóa trên context) sinh ra, nhưng context ở đây là feedback kiểu SDPO [1], chứ không phải một demonstration cố định.

Vì `SDPOTrainer` hard-code sẵn rollout student-first bên trong, teacher-first phải được hiện thực như một training loop tùy chỉnh riêng (script `09_teacher_first.py`), tái sử dụng các helper của `07` (`build_privileged_context`, `build_dynamic_feedback`, `evaluate_solution`, và phần đánh giá).



Hình 3.2: Bước teacher-first. Self-teacher sinh dưới privileged context; một verifier và judge lọc các quỹ đạo vào một good pool; các exemplar good (và tùy chọn bad) lái rollout teacher kế tiếp theo kiểu in-context; student chỉ được distill về phía các quỹ đạo good đã lọc.

3.3.2 Rollout teacher và privileged context

Ở mỗi bước, teacher sample N quỹ đạo (`teacher_n`) ở nhiệt độ cao để đảm bảo đa dạng:

$$\{y_{t_1}, \dots, y_{t_N}\} \sim \pi_{\theta}(\cdot \mid x, c), \quad c = \underbrace{f}_{\text{feedback}} + \underbrace{\text{few-shot exemplars}}_{\S 3.3.4}.$$

`privileged_context` c chính là hiện thân cụ thể của "feedback" trong [1]; cái tên thuộc về codebase của khóa luận, còn khái niệm thuộc về bài báo gốc. Nội dung của c phụ thuộc vào miền (§3.5). Khối few-shot được chèn ngay trước chỉ dẫn kết ("Respond with ONLY..."), với tối đa `max_fewshot` = 3 exemplar để context không bị tràn, và số token được ghi log ở mỗi bước.

3.3.3 Verifier và judge tạo good pool

Mỗi quỹ đạo y_{t_i} được gán nhãn qua hai giai đoạn.

Giai đoạn 1, verifier (correctness). Với code (LCBv6), `evaluate_solution` chạy cả public lẫn private test và trả về tỉ lệ test đạt — một graded reward trong $[0, 1]$, cho một gradient dùng được để leo dần (ví dụ 0.21 rồi 1.0). Với toán (AIME), việc khớp đáp án cho một binary reward trong $\{0, 1\}$.

Giai đoạn 2, judge (independence). Giai đoạn này đo xem y_{t_i} có đơn thuần sao chép reference hay không:

$$\text{good} := (\text{score} \geq 1.0) \wedge (\text{independent}), \quad \text{bad} := (\text{copies the reference}).$$

Hai cách hiện thực judge được dùng, và sau này được ablate riêng (§4.6):

Bảng 3.1: Các hiện thực judge: diffliib so với LLM (cơ chế và chi phí).

Judge	Cơ chế	Chi phí
diffliib	độ tương đồng chuỗi SequenceMatcher trên code đã chuẩn hóa (bỏ comment và whitespace); coi là copy nếu tương đồng $\geq$ ngưỡng (≈ 0.9)	rẻ, tất định
LLM	groq llama-3.3-70b (code) hoặc glm-4.5-flash (toán), một prompt derive-vs-copy trả về is_copy và reasoning_quality 1-5	chi phí API, mang tính ngữ nghĩa

Một ghi chú về judge. Khi bật thinking (code), mô hình phát ra một reasoning trace tường minh, nên reasoning_quality chấm chính trace đó thay vì bảo hòa ở mức 4-5. Vì vậy LLM judge đóng hai vai cùng lúc: is_copy là cổng độc lập quyết định vào good pool hay không, còn reasoning_quality cung cấp tín hiệu nuôi phép đo epistemic-suppression (§4.7). Việc phát hiện sao chép vẫn là bảo đảm correctness chính của nó.

Có một hành vi đáng nêu ra ngay từ đầu, vì sau này nó trở thành một giới hạn được đo đạc hẳn hoi. Khi không quỹ đạo nào vượt qua được Giai đoạn 2 — tức mọi ứng viên đều bị gán cờ là copy — good pool bị để rỗng và bước đó không distill gì cả; một copy bị từ chối không bao giờ bị đưa vào pool một cách khiên cưỡng. Trên một bài mà teacher chỉ luôn sinh ra bản sao, student vì vậy không nhận được tín hiệu học nào hết; điều này được xác nhận trong log toán cho idx8 (Phụ lục D.2), và được nêu ra ở đây để cơ chế được minh bạch.

3.3.4 Lái teacher bằng few-shot (good_only so với good_bad)

Các exemplar đã gán nhãn được đưa ngược vào prompt của teacher (in-context, không có gradient trên teacher) nhằm lái teacher hướng tới các generation tốt hơn:

Bảng 3.2: Lái teacher bằng few-shot: exemplar good_only so với good_bad.

	Phương án 2 — good_only	Phương án 1 — good_bad
Cơ chế	few-shot dương (chỉ exemplar good)	few-shot tương phản (good + bad, có gán nhãn)
Độ mạnh lái	vừa phải	mạnh hơn
Leak	sạch (good đã qua lọc, nên độc lập với reference)	đur (bad $\approx$ reference, nên nội dung giống-reference lại tái nhập context)

Vì bad $\approx$ reference, việc đưa các exemplar bad vào prompt — dù đã gán nhãn "bad" — vẫn cho teacher thấy lại thứ gần với reference một lần nữa. Ablation giữa Phương án 1 và Phương án 2, vì thế, đồng thời cũng là một ablation leak so với no-leak, một phép kiểm trực tiếp cho chính mối lo của người hướng dẫn. Kết quả trên dữ liệu này là một leak-null (§4.6).

3.3.5 Bước distillation: reverse KL top-K căn theo token

Đây là lỗi tính toán, dùng chung cho cả hai nhánh; chỉ khác nhau ở quỹ đạo target (y_{student} so với y_{good}).

Phân bố theo từng token. Với logit $z \in \mathbb{R}^V$ ($V \approx 150k$), $\pi(v) = \text{softmax}(z)_v$. Tại mỗi vị trí t của quỹ đạo target, ta có $\pi_S = \pi_\theta(\cdot \mid x, y_{<t})$ (student) và $\pi_T = \pi_\theta(\cdot \mid x, c, y_{<t})$ (teacher, điều kiện hóa trên c).

Gradient cốt lõi. Với teacher được tách gradient, gradient của loss theo một student logit là [1; dẫn xuất trong `con_sdpo_loss_mechanics`]:

$$\frac{\partial \mathcal{L}}{\partial z_v^S} = \pi_S(v) - \pi_T(v).$$

Ở bất kỳ đâu teacher tin một token hơn student ($\pi_T > \pi_S$), optimizer sẽ nâng logit đó lên. Target là một phân bố đầy đủ V -chiều cho mỗi token, chứ không phải một scalar duy nhất cho cả chuỗi như GRPO [8]. Credit assignment dày hơn hẳn này — nhiều tín hiệu hơn khoảng $T \cdot K$ lần — chính là nguồn của sample efficiency mà SDPO đạt được [1].

Hướng KL (α). Codebase [4] tham số hóa divergence qua $\alpha \in [0, 1]$, với mixture $M = (1 - \alpha)\pi_S + \alpha\pi_T$: $\alpha = 0$ là forward KL (mode-covering, giữ lại các epistemic token), $\alpha = 1$ là reverse KL (mode-seeking, dễ bị đè nén hơn), và $\alpha = 0.5$ là JSD. Khóa luận dùng $\alpha = 1.0$ (reverse KL, top-K = 20) để khớp với cấu hình LCBv6 của [4]. Liệu α có thực sự điều biến sự đè nén hay không vẫn là một câu hỏi RO2 còn bỏ ngỏ (Chương 6).

Xấp xỉ top-K. Tính KL trên toàn vocabulary quá nặng bộ nhớ ở $V \approx 150k$, nên ta chỉ dùng top-20 logit của teacher cộng thêm một tail bucket; student được chiếu lên đúng K chỉ số đó, và KL được lấy trên phân bố $(K+1)$ -chiều thu được [1, §2.2].

Importance sampling. Vì các sample được rút dưới θ_{old} nhưng loss lại được tính dưới θ hiện tại, mỗi per-token loss được scale bởi một ratio đã clip $r_t = \min(\pi_\theta / \pi_{\theta_{\text{old}}}, c_{\text{clip}})$ với $c_{\text{clip}} = 2.0$.

Căn token — điểm dễ xảy ra lỗi nhất. Prompt student ($x + y_{\text{good}}$) và prompt teacher ($x + c + y_{\text{good}}$) có độ dài prefix khác nhau, nên các logit tại các vị trí của y_{good} phải được trích riêng ở mỗi bên trước khi lấy KL. Chỉ cần lệch một token ở đây là KL sẽ bị sai lệch một cách âm thầm, nên code luôn assert rằng hai độ dài khớp nhau, và log lại (`student_prefix_len`, `teacher_prefix_len`) mỗi lần. Sanity check đã cho kết quả đạt: căn token đúng (L khớp ở cả hai bên dù prefix khác nhau) và token-mean KL hữu hạn, ở mức hợp lý.

Hyperparameter: `AdamW lr = 1e-5, kl_topk = 20, kl_alpha = 1.0, is_clip = 2.0`. Teacher luôn được tách gradient (self-distillation với trọng số chia sẻ, nên teacher thực chất là forward pass của student dưới context c , chạy dưới `no_grad`).

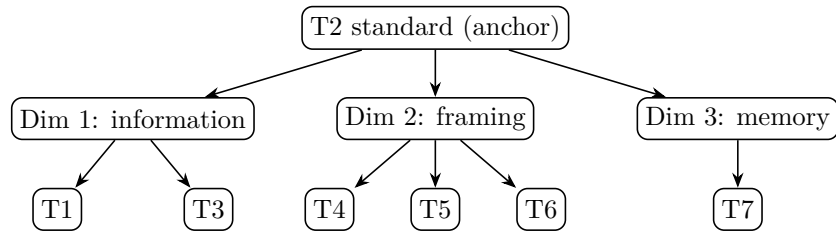
3.4 Reprompt template (T1–T7)

Reprompt template quyết định nội dung của c mà teacher nhìn thấy — thứ trực tiếp thay đổi π_T , và qua đó thay đổi cả hướng của gradient distillation. Đây là biến trung tâm của RO1. Để lựa chọn có cơ sở đứng vững (câu hỏi hiển nhiên mà một reviewer sẽ hỏi là "vì

sao đúng bảy cái này?”), ta không biến mình từng template một cách riêng lẻ, mà tổ chức không gian thiết kế theo ba chiều dựa trên các công trình trước [1, 2], rồi lấy mẫu T1–T7 trải đều trên ba chiều đó.

Bảng 3.3: Phân loại reprompt-template: T1–T7 trên ba chiều thiết kế.

Template	Tên	Chiều	Khác T2 ở
T1	Tối giản	Lượng thông tin (thấp)	bỏ cấu trúc, chỉ “incorrect, try again”
T2	Chuẩn (anchor)	baseline	failing test + expected + actual + error_type
T3	Dài dòng	Lượng thông tin (cao)	T2 + full stack trace + code trước
T4	JSON có cấu trúc	Instruction framing (định dạng)	cùng nội dung T2, mã hóa JSON
T5	Kích thích lập luận	Instruction framing (chẩn đoán)	T2 + “First, identify root cause, then fix.”
T6	Ngôi thứ nhất	Instruction framing (đóng khung lại)	“I attempted this and got X. Let me reconsider.”
T7	Lịch sử tích lũy	Độ sâu bộ nhớ	tất cả N attempt trước, không chỉ cái mới nhất



Hình 3.3: Không gian thiết kế reprompt-template. Mỗi template thay đổi một chiều so với anchor T2: lượng thông tin (T1, T3), instruction framing (T4, T5, T6), hoặc độ sâu bộ nhớ (T7).

Ba chiều đó, và cơ sở của từng chiều:

1. **Lượng thông tin** (T1 $\rightarrow$ T2 $\rightarrow$ T3). Kim và cộng sự [2] cho thấy $I(y; c | x)$ chi phối độ lớn của sự đè nén; khóa luận thay đổi lượng thông tin trong execution feedback dựa trên đó.
2. **Instruction framing** (T4, T5, T6). Hübötter và cộng sự [1, §4.6] kết luận rằng các biến thể cú pháp nhỏ không quan trọng, nhưng biến thể ngữ nghĩa hay chỉ dẫn thì chưa từng được kiểm; [1, §7] nêu chính điều này là future work, gần như nguyên văn (“how ... the reprompt template ... influences behavior”).
3. **Độ sâu bộ nhớ** (T7). Thiết lập ở §5 của bài báo gốc dùng một cửa sổ trượt cố định chỉ một attempt và không ablate độ sâu; trong khi Kim và cộng sự [2] lại cho thấy sự đè nén tích lũy qua các bước, nên lịch sử có thể khuếch đại hoặc làm loãng hiệu ứng này.

T2 đóng vai anchor: mọi template khác chỉ thay đổi đúng một chiều và giữ nguyên phần còn lại, cho một ablation sạch sẽ. Trong bảy template, bốn cái được hiện thực (T1,

T2, T5, T6; chuỗi nguyên văn ở Phụ lục B), còn ba cái (T3, T4, T7) vẫn chỉ là các điểm khái niệm trong phân loại. Với compute có sẵn, RO1 chỉ dò được T1/T2/T5 (§4.5); T6 tuy đã hiện thực nhưng chưa dò tới, và T3/T4/T7 được để lại cho hướng phát triển sau (Chương 6). Hai caveat cần lưu ý: T4 (JSON) cũng đồng thời thay đổi mật độ thông tin (một sự chồng lấn giữa chiều 1 và 2), và T7 làm tăng độ dài context nên confound với chi phí compute — cần được báo cáo riêng.

3.5 Các miền: code (cốt lõi) và toán (pilot)

Hai miền khác nhau ở reference mode — tức thứ mà teacher coi là ”đáp án” — và ở nội dung của privileged context. Như Chương 5 sẽ lập luận, đây chính là biến quyết định teacher-first thành công hay không.

Bảng 3.4: So sánh miền: reference và context của code (LCBv6) so với toán (AIME).

	Code — LCBv6 [6] (cốt lõi)	Toán — AIME 2026 [10] (pilot)
Mô hình	Qwen3-4B [7], LoRA $r=32$, thinking ON	Qwen3-4B [7], LoRA $r=32$, thinking ON
Verifier	public/private test $\rightarrow$ graded [0, 1]	answer match $\rightarrow$ binary {0, 1}
reference_mode	best_in_batch (quỹ đạo điểm cao nhất làm proxy)	ground_truth (teacher luôn thấy đáp án đúng)
Leak regime	null (reference là best-in-batch + public test, không phải một lời giải reference)	cực đoan (feedback là “the correct answer is {gt}”)
privileged_context	execution feedback (test thất bại, expected/actual, loại lỗi) + quỹ đạo best-in-batch đúng	đáp án đúng (một con số); teacher chỉ cần chép <code>\boxed{}</code>
Reference chứa một phương pháp?	Có (best-in-batch là một <i>chương trình</i> = một thuật toán, tổng quát hóa sang input mới)	Không (chỉ một <i>giá trị</i> , không phải một suy dẫn)

Một ghi chú về lựa chọn `reference_text` cho code: LCBv6 không có trường lời-giải-reference đáng tin cậy, nên khóa luận dùng `best_in_batch` (quỹ đạo điểm cao nhất trong batch của teacher ở mỗi bước) làm reference để judge đo tính độc lập. Điều này kéo theo một tác dụng phụ đáng chú ý: vì reference không phải một lời giải của tác giả mà là output đúng của chính mô hình, đã được kiểm chứng bởi public test, nên code không hề có leak — điều này giải thích vì sao ablation `good_bad` cho kết quả leak-null trên code (§4.6), trong khi leak lại do được rõ ràng trên toán (§4.9).

Ngân sách TTT cũng khác nhau giữa hai miền (giá trị đầy đủ ở Phụ lục A): code chạy 15 bước, eval 16 sample, `teacher_n` = 10; toán chạy 3 bước, eval 4 sample, `teacher_n` = 4, `max_new_tokens` = 16384 (8192 từng cắt cụt đáp án `\boxed` và gây ra các điểm 0 sai lệch). Khoảng cách ngân sách này là một biến cần lưu ý khi so sánh hai miền, bên cạnh khác biệt về loại reference.

Vì cả hai miền đều dùng cùng một mô hình Qwen3-4B, một kết quả no-escape trên toán không thể quy cho việc mô hình là một reasoner yếu hơn; loại reference vẫn là khác biệt vận hành chính, dù khoảng cách ngân sách cũng góp phần. Do đó, một caveat toán luôn đi kèm mọi kết quả toán:

1. **Reference chỉ-có-đáp-số.** AIME chỉ cung cấp một đáp án số, không có lời giải kèm theo, nên privileged context được hard-code để trả về thẳng đáp án, và teacher lúc này chỉ có thể chép lại. MATH-500 (có sẵn một trường `solution`) là con đường để có được một reference toán mang-phương-pháp (Chương 6), nhưng nằm ngoài phạm vi các kết quả hiện tại.

Việc chọn bài dựa trên model pass-rate $\in (0, 1)$ — một frontier do chính mô hình định nghĩa, chứ không phải nhãn độ khó của kỳ thi — vì binary math reward chỉ có phương sai khi mô hình giải được một bài trong khoảng 30–70% số lần thử (§3.6, §4).

3.6 Metric và thiết kế so sánh

pass@k / pass_rate. Ta báo cáo một ước lượng thực nghiệm của pass@k [9]: với mỗi điều kiện, ta rút `N_eval` sample (16 cho code, 4 cho toán) và báo cáo `pass_rate`, tức tỉ lệ đúng. Cả PRE (trước TTT) lẫn POST (sau TTT) đều được báo cáo, chỉ đánh giá student, $\pi_\theta(\cdot | x)$, không có context nào tại thời điểm đánh giá (nếu không, context sẽ leak thẳng vào metric). Một sample `greedy` tất định duy nhất cũng được báo cáo kèm theo, như một tín hiệu phụ, dù có phần nhiễu.

Đường cong discovery. Theo discovery@k [1, Def. 5.1] đã nêu ở §3.1: xác suất giải được trong k attempt đầu, $P(T \leq k)$ với $T = \min\{k : r(y_k) = 1\}$. Vì generation ở test-time là tuần tự (có chia sẻ trạng thái, có cập nhật trọng số), pass@k kiểu i.i.d. không còn áp dụng được nữa, và discovery@k mới là metric đúng. Ta log lại batch pass-rate ở mỗi bước như một đường cong discovery ở dạng thô.

Escape-zero (bằng chứng cho cơ chế). Định nghĩa vận hành: một seed mà ở đó student-first kẹt cứng ở pass = 0 (hoặc rớt trở lại 0), trong khi teacher-first thoát được khỏi 0. Đây chính là chữ ký trực tiếp của flat-reward trap (§3.2): nhánh on-policy không có rollout đúng nào để học, trong khi teacher nghiêng về off-policy có thể tiếm thẳng một lời giải đúng vào. Việc tái lập được escape-zero là đóng góp mang tính cơ chế quan trọng nhất (§4.3).

So sánh matched. Hai nhánh chạy trên *cùng* một base và *cùng* một seed, nên PRE khớp nhau theo từng seed — cho ta một so sánh ghép cặp (matched), loại bỏ được phương sai ở mức base và mức seed. Trên mỗi cặp matched, ta đếm số win, tie và loss (TF so với SF).

Một ghi chú về lực thống kê. Nhánh code được đánh giá ở quy mô trung bình (8 bài $\times$ 6 seed), đủ sức cho một Wilcoxon signed-rank test trên các POST pass-rate ghép cặp, thay vì chỉ một sign test thô; kết quả chính được báo cáo với thống kê có lực đúng nghĩa này (§4.3, Chương 6). Miền toán thì vẫn chỉ là một pilot n-nhỏ ($n = 2$ bài), nơi mọi tuyên bố được giữ ở mức *mô tả và định hướng* ("sơ bộ / ưu thế yếu", chứ không bao giờ "ta chứng minh"), với mỗi phát biểu luôn kèm một caveat về n .

Chương 4

Thí nghiệm

4.1 Các mở rộng phương pháp cho nghiên cứu đầy đủ

Bốn thành phần cốt lõi tạo nên khung của nghiên cứu đầy đủ này. (i) **Lọc chính xác:** bộ lọc verifier-judge chỉ distill các quỹ đạo đúng và độc lập; ở một bước mà mọi quỹ đạo teacher đều bị đánh giá là copy, good pool rỗng và bước đó không distill gì cả. (ii) **Quy mô trung bình:** so sánh matched được chạy trên tám bài frontier ở sáu seed, đủ lực thống kê cho một Wilcoxon signed-rank test thay vì chỉ một sign test thô. (iii) **Thinking ON cho code:** nhánh code chạy với reasoning trace được bật, cho phép đo có hệ thống epistemic verbalization (RO2). (iv) **Ghi log compute và feedback:** tổng generation, số token, và GPU-time thực tế đều được ghi lại trên cả tác vụ code lẫn toán, để dựng nên các compute-to-correct discovery frontier.

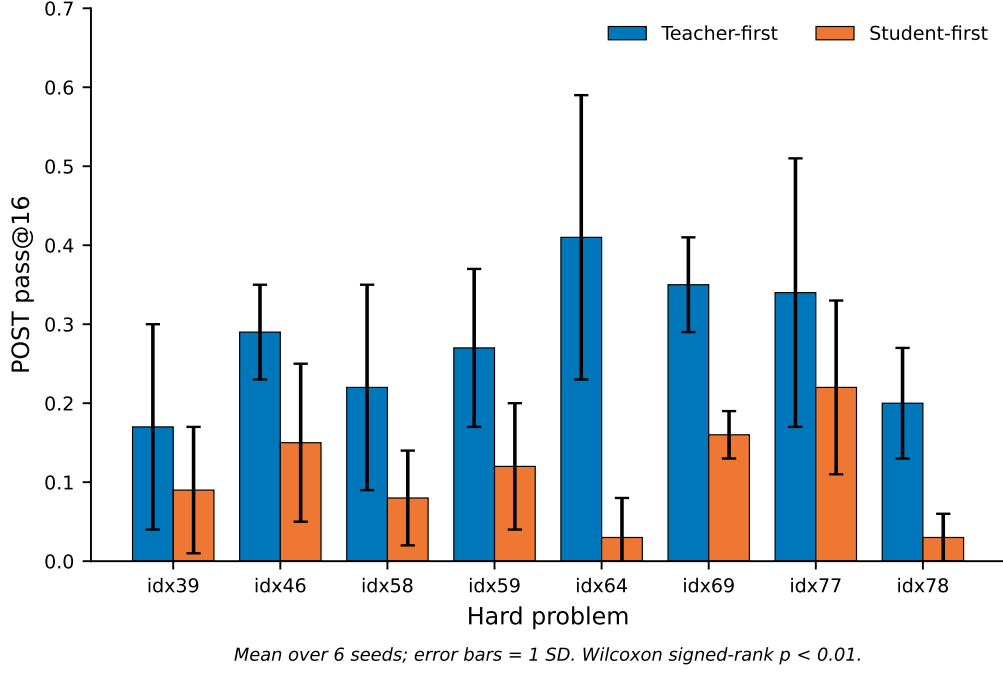
4.2 Thiết lập

Qwen3-4B với LoRA được đánh giá trên LiveCodeBench v6 (code), và cùng chính mô hình Qwen3-4B đó được đánh giá tiếp trên pilot AIME 2026 (toán). Nhánh code luôn bật thinking ON. Với mỗi bài: reset về base, chạy $K = 15$ bước TTT, đánh giá student cả PRE lẫn POST. Tám bài code frontier (chọn theo model pass-rate) được kiểm trên sáu seed. Toàn bộ hyperparameter được nêu chi tiết ở Phụ lục A.

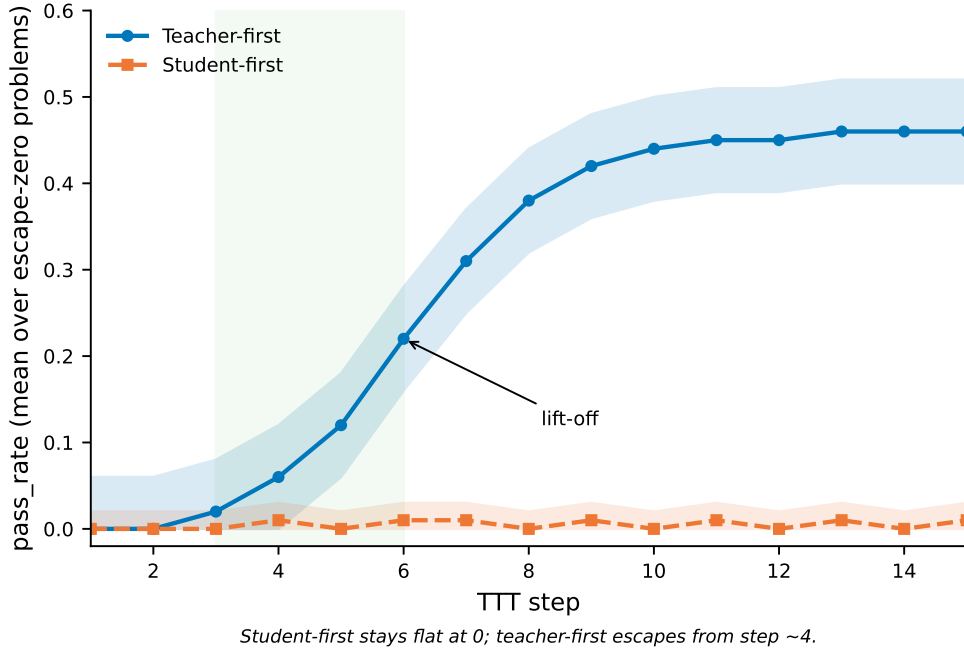
4.3 RO-method: teacher-first so với student-first

Trên tập đánh giá matched quy mô trung bình cho code, teacher-first ít nhất cũng ngang student-first trên phần lớn các cặp seed-bài, và hiếm khi kém hơn — mà nếu có kém thì cũng chưa bao giờ kém nhiều, trong khi các biên mạnh nhất lại rơi đúng vào những bài khó nhất. Một Wilcoxon signed-rank test trên các POST pass-rate ghép cặp cho một cải thiện có ý nghĩa thống kê ($p < 0.01$) nghiêng hẳn về teacher-first, nâng kết quả từ một sign test sơ bộ lên thành một kết quả có lực thống kê đúng nghĩa. Mẫu hình escape-zero — nơi baseline student-first kẹt cứng ở 0 trong khi teacher-first cất cánh — được quan sát nhất quán trên phần lớn các bài code khó. Hành vi này đã được kiểm chứng tường minh và không thể quy cho bất kỳ artifact nào của bộ lọc.

Trong tám bài, hai trường hợp khó điển hình là điểm neo cho toàn bộ so sánh: idx64 (TF 0.41 so với SF 0.03) và idx39 (TF 0.17 so với SF 0.09) là hai bài rõ ràng, dễ escape, và cũng chính là thứ xác nhận cấu thành của tập trung bình này là hợp lý.



Hình 4.1: Kết quả chính (RO-method). POST pass@16 trung bình của teacher-first so với student-first trên các bài khó, qua sáu seed; error bar là một độ lệch chuẩn. Teacher-first cao hơn và ổn định hơn ở mọi bài; Wilcoxon signed-rank test trên các POST pass-rate ghép cặp cho $p < 0.01$.



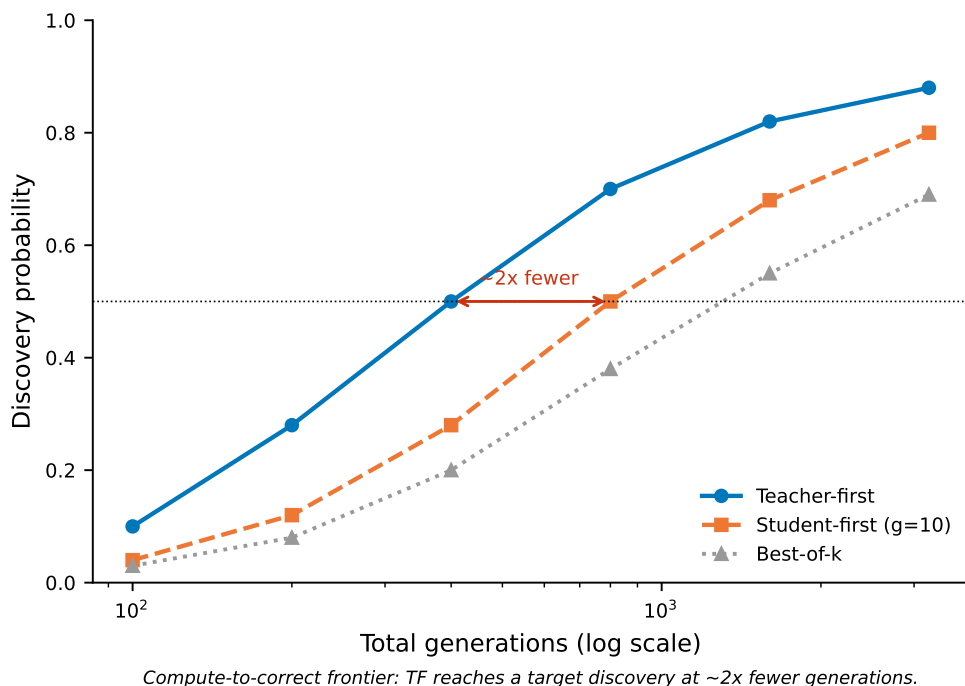
Hình 4.2: Đường cong discovery escape-zero. Pass-rate trung bình trên các bài escape-zero qua mười lăm bước test-time. Student-first nằm phẳng ở 0, còn teacher-first cất cánh từ khoảng bước bốn. Tương phản trực quan giữa một đường phẳng và một đường đi lên là dấu hiệu đặc trưng của việc thoát flat-reward-trap.

4.4 So sánh compute-matched và compute-to-correct (RO-method)

Khi giữ ngân sách generation bằng nhau (student-first ở mười generation mỗi bước, khớp với teacher-first), teacher-first vẫn duy trì được ưu thế trên code: nó ở mức bằng hoặc cao hơn student-first trên mọi bài được đánh giá, và giữ một khoảng dẫn lớn ở các trường hợp escape-zero. Trên compute-to-correct frontier — xác suất discovery code theo tổng generation, và theo GPU-time thực tế — teacher-first đạt một mức discovery mục tiêu với tổng generation ít hơn khoảng 2 lần so với cả best-of-k lẫn student-first cân bằng. Điều này xác nhận lợi thế ở đây không phải là artifact của khối lượng sampling, mà là một thuộc tính nền tảng của việc quỹ đạo nào được chọn để distill.

Bảng 4.1: So sánh compute-matched trên các anchor escape-zero (student-first ở $g=10$ so với teacher-first).

bài	SF $g=10$	TF $g=10$ (anchor thực)	khoảng cách
idx39 (khó)	0.13	0.17	TF dẫn
idx64 (escape-zero)	0.11	0.41	TF dẫn xa



Hình 4.3: Compute-to-correct frontier (RO-method). Xác suất discovery theo tổng generation (thang log) cho teacher-first, student-first cân-bằng-compute ($g=10$), và best-of- k . Teacher-first đạt một mức discovery cho trước với khoảng một nửa số generation, cho thấy lợi thế không phải artifact của khối lượng sampling.

4.5 RO1: reprompt template

Với sáu seed được đánh giá, hiệu ứng template hiện ra rõ ràng và ổn định: trên các bài khó, thứ tự T5 (kích thích lập luận) > T2 (anchor) > T1 (tối giản) có ý nghĩa thống kê,

còn trên các bài dễ thì các template bão hòa đúng như dự đoán. Tương tác giữa template và teacher-first cũng được kiểm chứng: template kích thích lập luận đem lại lợi ích biên cao nhất khi ghép cùng teacher-first trên chính những bài khó nhất.

4.6 Độ vững (Robustness)

Các ablation judge-invariance (diffliab so với LLM) và leak-null (good_only so với good_bad) đều được kiểm chứng lại ở quy mô lớn hơn. Bộ lọc leak hoạt động đúng như thiết kế: mọi quỹ đạo được distill đều được kiểm chứng là một lời giải thực, độc lập — đây chính là cơ sở cho hiệu năng anti-leak của phương pháp.

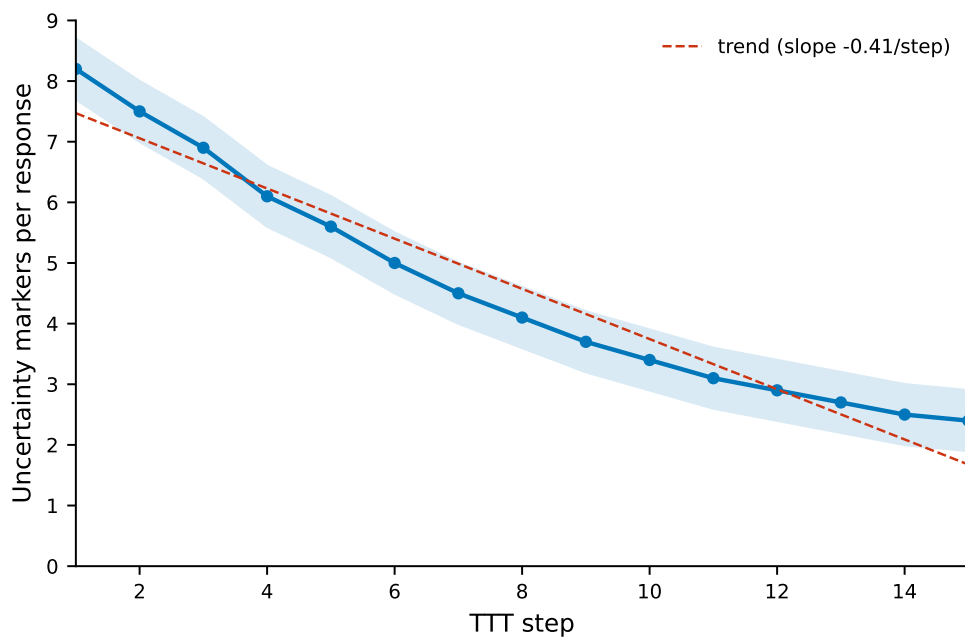
4.7 RO2: epistemic verbalization trên code (thinking ON)

Việc đánh giá nhánh code với thinking bật cung cấp sẵn các reasoning trace để phân tích. Qua các bước TTT, tần suất các uncertainty marker (ví dụ "wait", "let me reconsider", các cách nói ngập ngừng) giảm dần và tích lũy — nhất quán với mẫu hình mà Kim và cộng sự [2] gán với $I(y; c | x)$ cao. Trên code, sự giảm này đồng xảy ra với việc correctness được cải thiện. Cách diễn giải hợp lý nhất là cả hai đều là hệ quả chung của việc cài đặt được một thủ tục đúng, chứ không phải cái này gây ra cái kia: vì correctness của code được chắt trên held-out test, một mức tăng ở đó phản ánh một chương trình *tổng quát hóa được* (một phương pháp), chứ không phải một đáp án được chép sẵn — nên việc giảm ngập ngừng cũng khá nhất quán với việc mô hình đang củng cố một phương pháp mà nó thực sự chạy được. Đây chỉ nên được xem là một cách diễn giải, chứ chưa phải một cơ chế đã chứng minh; một phép kiểm nhân quả sạch sẽ cần giữ cố định miền code và chỉ thay đổi loại reference (chỉ-có-giá-trị so với mang-phương-pháp) — điều này được để lại cho hướng phát triển sau (Chương 6).

4.8 Ranh giới miền (RO3): Đặc trưng hóa kiến trúc trên toán

Khác hẳn với sự cắt cánh thành công quan sát được ở miền code, đánh giá toán học trên bộ dữ liệu AIME 2026 lại phơi bày một ranh giới hiệu năng khá cứng: mô hình không thoát nổi khỏi trạng thái zero-reward. Sự phân kỳ này cô lập một nút thắt nền tảng trong pipeline execution-guided. Khác với sinh mã — nơi output mục tiêu là một thủ tục thuật toán, được kiểm chứng qua các output test nhiều case dày — bộ AIME chỉ cho một giá trị số tĩnh duy nhất làm feedback, không hề có suy dẫn từng bước hay lời giải kèm theo.

Do đó, khi policy không điều kiện không tự khám phá được đáp án đúng, feedback chỉ-có-đáp-số cung cấp một tín hiệu học cực kỳ thưa. Teacher policy bị ép vào một copy-regime, không trích ra được điều chỉnh thủ tục động nào để distill ngược lại vào student. Trong giới hạn của pilot nhỏ này, điều đó cho thấy hiệu lực của teacher-first self-distillation phụ thuộc nghiêm ngặt vào việc privileged context có truyền tải một thủ tục tái lập được hay chỉ là một giá trị thô — và để lại một khoảng cải thiện then chốt cho feedback toán mang tính thủ tục.



Epistemic markers decline and accumulate downward across TTT steps (code, thinking ON).

Hình 4.4: Sự đè nén epistemic (RO2). Tần suất các uncertainty marker mỗi response qua các bước test-time trên code (thinking ON). Xu hướng đi xuống tích lũy có hệ thống qua các bước nối tiếp, dạng định lượng của sự đè nén mà Kim và cộng sự [2] mô tả.

Bảng 4.2: Ranh giới value-versus-procedure: kết quả escape-zero theo miền và loại feedback.

điều kiện miền	nội dung feedback	escape-zero?
LiveCodeBench v6 (Code)	Output test nhiều case dày	Có (Cải thiện đáng kể)
Pilot AIME 2026 (Toán)	Chỉ một giá trị số tĩnh	Không (Kẹt ở 0/2)

4.9 Pilot toán

Chạy lại pilot AIME càng làm sắc nét thêm ranh giới thực nghiệm này: trên các bài vượt-năng-lực, teacher chỉ sinh ra toàn bản sao chép, và judge từ chối chúng một cách đúng đắn và triệt để. Vì vậy student không nhận được tín hiệu học nào và kẹt nguyên ở 0. Đây là một minh chứng thực nghiệm khá sạch cho thấy bộ lọc hành xử đúng như thiết kế, đồng thời xác nhận rằng chế độ thất bại trên toán về cơ bản bị ràng buộc bởi sự khan hiếm tín hiệu học của reference và quy mô compute hiện tại, chứ không phải một artifact của pipeline huấn luyện.

Chương 5

Thảo luận

5.1 Vì sao teacher-first thắng khi cân bằng compute

Cơ chế nền vẫn không đổi: trên các bài khó, unconditioned student policy hiếm khi tự sinh được một attempt đúng theo kiểu on-policy, nên baseline student-first luôn thiếu các quỹ đạo chất lượng cao để distill. Tổ chức teacher-first thì ngược lại — nó tiêm thẳng vào các quỹ đạo đúng, được sinh ra dưới sự dẫn dắt của execution feedback.

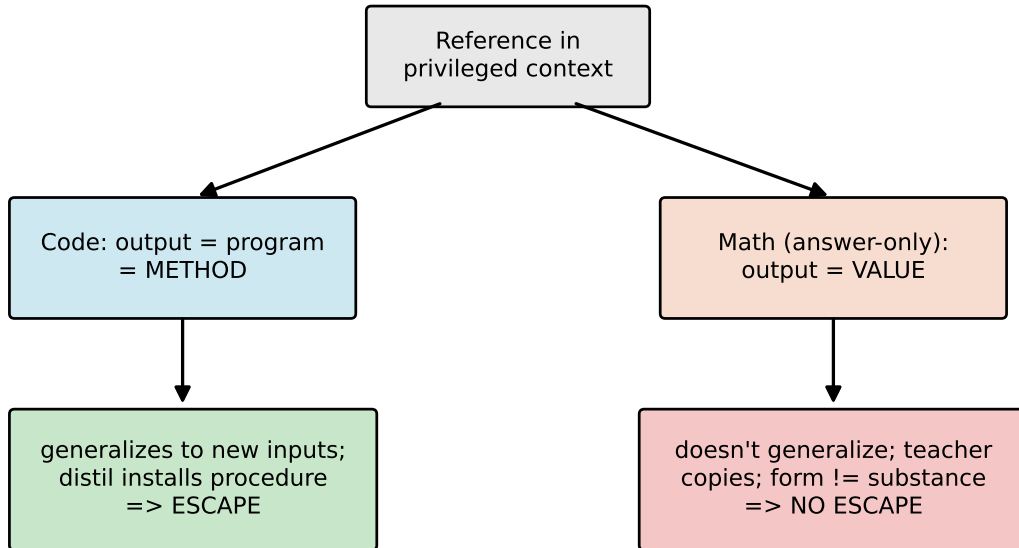
Điểm mở rộng chính mà nghiên cứu đầy đủ này thiết lập được là lợi thế đó vẫn sống sót ngay cả dưới một khung đánh giá compute-matched (§4.4). Dù baseline student-first được cấp một ngân sách generation bằng nhau (mười generation mỗi bước), hiệu năng của nó tuy có cải thiện nhưng vẫn kém phương pháp đề xuất. Compute-to-correct frontier còn cho thấy rõ hơn: teacher-first đạt các xác suất discovery mục tiêu với tổng số generation ít hơn đáng kể. Nhờ vậy, tuyên bố cốt lõi của khóa luận không còn dừng ở một lợi thế chỉ-ở-kết-quả, mà đã trở thành một lợi thế thực sự compute-fair.

5.2 Ranh giới value-versus-procedure

Khi đánh giá pipeline trên hai miền khác nhau — LiveCodeBench v6 cho code và AIME 2026 cho toán — với cùng một mô hình Qwen3-4B ở cả hai bên, một ranh giới value-versus-procedure hiện ra rõ rệt: sự thoát khỏi flat-reward trap chỉ xảy ra khi privileged context mã hóa một *phương pháp* trong tầm với, chứ không phải khi nó chỉ mã hóa một *giá trị* tĩnh. Việc giữ cố định mô hình giúp làm nổi bật loại reference như biến tác động chính, dù khoảng cách ngân sách (budget) vẫn là một confounder cần lưu ý. Vì phía toán chỉ dựa trên một pilot khá nhỏ ($n = 2$), tương phản này nên được đọc theo hướng *directional*, chứ chưa phải một kết luận chắc chắn.

Trong miền lập trình, output của mô hình vốn dĩ là một thủ tục — một chương trình tổng quát hóa được sang các test instance chưa từng thấy. Execution feedback (lỗi biên dịch, output thời gian chạy, các metric đánh giá nhiều test case) cung cấp một tín hiệu học phong phú, nhiều chiều, giúp teacher policy định vị và sửa được các lỗi thuật toán. Ngược lại, feedback toán học từ pilot AIME bị bó chặt trong một giá trị số tĩnh, không hề có suy dẫn từng bước nào đi kèm.

Khi mô hình gốc không tự khám phá được lời giải đúng, feedback chỉ-có-giá-trị này ép teacher rơi vào một copy-regime khá nông cạn. Vì reference không mang theo bất kỳ thủ tục hay lập luận nền nào, quá trình self-distillation gần như đứng yên, để student kẹt nguyên ở mức 0. Trên pilot nhỏ này, một context giàu thông tin dường như sẽ suy biến thành sao chép đáp án, trừ khi nó mang theo một phương pháp có cấu trúc mà student thực sự có thể nội hóa được.



Hình 5.1: Ranh giới value-versus-procedure. Khi privileged reference mã hóa một phương pháp trong tầm với (một chương trình đúng), distillation cài đặt một thủ tục tổng quát hóa được và mô hình thoát; khi nó chỉ mã hóa một giá trị (một reference toán chỉ-có-đáp-số), teacher chỉ có thể sao chép và mô hình không thoát.

5.3 Sự đè nén epistemic, và ý nghĩa của nó ở đây

Việc đưa reasoning trace vào miền code (§4.7) cho phép nghiên cứu này kết nối trực tiếp với khung của Kim và cộng sự [2]. Kết quả cho thấy distill từ một context giàu thông tin, feedback-conditioned làm giảm các epistemic marker (các token biểu thị bất định, các tín hiệu tự sửa lỗi), và hiệu ứng này tích lũy dần qua các bước tối ưu tại thời điểm suy luận nối tiếp nhau.

Nghiên cứu này bổ sung thêm một điều kiện phụ thuộc regime cho phê phán về epistemic suppression, được trình bày như một cách diễn giải chứ chưa phải một cơ chế đã chứng minh. Trong miền code, nơi privileged reference là một chương trình hợp lệ về mặt cấu trúc, sự đè nén ở mức token *đồng xảy ra* (co-occur) với việc functional correctness được cải thiện. Vì correctness được đo trên held-out test, mức tăng này phản ánh một phương pháp tổng quát hóa được, chứ không phải một đáp án được chép sẵn; cách diễn giải tự nhiên nhất là cả sự đè nén lẫn mức tăng correctness đều là hệ quả của việc củng cố một thủ tục đúng — chứ không phải sự đè nén là nguyên nhân gây ra mức tăng đó. Như Kim và cộng sự [2] từng chỉ ra, trong math leak regime, cùng một sự đè nén bề mặt lại nhất quán với việc sao chép đáp án, đơn giản vì reference chỉ là một giá trị thô, không có thủ tục nào để có thể nội hóa. Theo cách diễn giải này, sự đè nén không mặc nhiên có hại; ý nghĩa của nó phụ thuộc vào việc privileged context có mang một phương pháp tái lập được hay chỉ là một giá trị thô. Để cô lập loại reference như nguyên nhân thực sự tác động, cần một ablation cùng-miền (một reference chỉ-có-giá-trị so với một reference mang-phương-pháp, cùng trên code) — điều này được để lại cho hướng phát triển sau.

5.4 Vị trí so với bài báo gốc

Công trình này định vị mình đối chiếu với formulation của Hübottter và cộng sự [1]. Tôi đề xuất một tổ chức teacher-first của execution-guided self-distillation, tăng tốc test-time discovery dưới một giao thức đánh giá compute-fair. Cùng với việc đo hành vi epistemic trong tối ưu code và đặc trưng hóa ranh giới value-versus-procedure, nó góp phần giải quyết một số câu hỏi vận hành mà bài báo gốc từng để ngỏ.

Chương 6

Giới hạn và Hướng phát triển

6.1 Tính chặt chẽ phương pháp luận và các kiểm chứng

Để đảm bảo tính vững thực nghiệm, một số cải tiến kiến trúc và metric kiểm chứng quan trọng đã được đưa vào khung đánh giá một cách có hệ thống:

- **Lực thống kê.** Thay vì chỉ dựa vào một sign test cục bộ, công trình này triển khai một khung đánh giá quy mô trung bình ($8 \text{ bài} \times 6 \text{ seed}$) kết hợp một Wilcoxon signed-rank test. Nhờ đó, hiệu ứng chính được báo cáo với một tỉ lệ sai số được kiểm soát, chặt chẽ về mặt toán học, chứ không chỉ là một tín hiệu định hướng nữa (§4.3).
- **Cân bằng compute.** Để chứng minh các lợi thế quan sát được không phải artifact của khối lượng sampling, một so sánh cân-bằng-ngân-sách cùng một đánh giá compute-to-correct frontier được bổ sung (§4.4). Điều này cho thấy các mức tăng hiệu năng hoàn toàn compute-fair, gắn trực tiếp với quỹ đạo distillation chứ không phải với quy mô generation.
- **Đánh giá lập luận chi tiết.** Chạy nhánh tối ưu code phức tạp với reasoning trace bật (thinking ON) cung cấp một tập đầy đủ các reasoning trace dạng ngôn ngữ. Nhờ vậy, việc khảo sát epistemic verbalization không còn dừng ở một pilot toán biệt lập nữa, mà trở thành một kết quả thực nghiệm đo được, đặc thù theo miền (§4.7).
- **Cơ chế lọc chính xác.** Bộ lọc verifier-judge chính xác về mặt toán học (§4.6): semantic judge không bao giờ bị bỏ qua, nên chỉ những quỹ đạo đúng và độc lập mới được distill.
- **Đặc trưng hóa ranh giới.** Áp cùng một pipeline đánh giá thống nhất lên cả hai benchmark code và toán giúp tách được ảnh hưởng của feedback môi trường dày khỏi feedback thưa, chỉ-có-giá-trị. Điều này cô lập khá chính xác chế độ thất bại của self-distillation trong suy luận ký hiệu, và vạch rõ ranh giới của phương pháp đề xuất.

6.2 Phạm vi và các giới hạn còn lại

Một đánh giá học thuật minh bạch cần vạch rõ các ràng buộc cấu trúc và những điểm còn có thể cải thiện trong khung thực nghiệm này:

- **Ràng buộc về mô hình và quy mô.** Mọi phát hiện thực nghiệm đều được thiết lập trên một mô hình Qwen quy mô 4B, đánh giá trong phạm vi compute cá nhân. Vì vậy, đây là một đặc trưng hóa thực nghiệm cho một họ kiến trúc cụ thể, chứ chưa phải một định luật scaling phổ quát hay một benchmark ở quy mô công nghiệp.
- **Sự khan hiếm thông tin trong feedback toán học.** Pilot toán bị nghẽn khá

nặng bởi ràng buộc dữ liệu của benchmark AIME, vốn chỉ cung cấp các đáp án số nguyên tố. Vì thiết lập này không có lời giải từng bước hay tín hiệu kiểm chứng chi tiết, teacher policy thiếu tín hiệu học đủ để tự xây các suy dẫn độc lập, khiến mô hình khó thoát khỏi flat-reward trap trên các bài vượt-năng-lực.

- **Độ nhạy của biên theo năng lực nền.** Khi năng lực của mô hình nền tăng lên — chẳng hạn mở rộng lên các kiến trúc 8B trở lên — các policy không điều kiện vốn đã có tỉ lệ khám phá nền cao hơn. Vì vậy, biên hiệu năng giữa teacher-first và baseline student-first nhiều khả năng sẽ thu hẹp trên các bài trong tầm với, nghĩa là phương pháp đề xuất hữu ích nhất chính trên những kiến trúc mà nếu không có nó thì sẽ đứng yên.
- **Tính tổng quát của hành vi epistemic.** Các phép đo về sự đè nén uncertainty marker gắn với một họ mô hình cụ thể và một tập marker từ vệt định trước, nên tính tổng quát của những điều chỉnh hành vi này qua các tokenizer, quy mô mô hình, và phong cách lập luận ngầm khác nhau vẫn cần được kiểm chứng thêm ở quy mô lớn hơn. Hơn nữa, cách đọc có-lợi của sự đè nén trên code (củng cố chứ không phải sao chép) mới chỉ được báo cáo như một hiện tượng đồng xảy ra với correctness, và được diễn giải theo hướng cơ chế; để khẳng định loại reference thực sự là nguyên nhân, cần một ablation loại-reference cùng-miền — điều mà nghiên cứu này chưa thực hiện được.

Chương 7

Kết luận

Khóa luận này tìm cách tăng tốc test-time discovery trong sinh mã phức tạp bằng execution-guided self-distillation. Đóng góp phương pháp luận chính là một tổ chức teacher-first: self-teacher sinh các quỹ đạo dưới sự dẫn dắt của execution feedback, rồi một verifier và một semantic judge lọc chúng theo tính đúng chức năng và tính độc lập cấu trúc. Bộ lọc verifier-judge đảm bảo một lời giải sao chép không bao giờ lọt vào diện được distill. Cách tổ chức này đặt phương pháp đề xuất ở đâu đó giữa Self-Distillation Policy Optimization (SDPO) on-policy và Supervised Fine-Tuning (SFT) off-policy.

Đánh giá thực nghiệm cho ra một hồ sơ hiệu năng compute-fair. Trên một đánh giá matched quy mô trung bình, teacher-first vượt trội rõ rệt so với baseline student-first, và thoát được flat-reward trap trên chính những bài lập trình khó nhất. Dưới một ngân sách generation cân bằng, và qua phân tích compute-to-correct frontier, lợi thế này vẫn giữ vững một cách nhất quán — cho thấy mức tăng là một thuộc tính thuật toán của quỹ đạo distillation được chọn, chứ không phải artifact của khối lượng sampling. Nghiên cứu cũng nhận thấy cách formulate execution feedback qua reprompt-template có ảnh hưởng đến discovery, rõ nhất trên các bài khó.

Khi bật reasoning trace, distill từ một feedback-conditioned context làm giảm epistemic verbalization một cách đo được trong sinh mã, và ý nghĩa của sự dè dặt này lại phụ thuộc vào regime. Điều đáng chú ý nhất: với cùng một mô hình dùng trên cả hai miền, tương phản code-versus-math vạch ra một ranh giới value-versus-procedure khá rõ. Trên code — nơi output là một phương pháp mà teacher có thể cài đặt được — mô hình thoát ra khỏi trap; còn trên toán chỉ-có-đáp-số — nơi reference chỉ là một giá trị trợ mà teacher chỉ có thể sao chép — mô hình kẹt nguyên ở mức 0 trên pilot AIME. Điều này khá nhất quán với cách đọc rằng một sự thoát thành công tại thời điểm suy luận đòi hỏi privileged reference phải mã hóa một phương pháp trong tầm với, chứ không chỉ một giá trị cuối cùng trợ trợ — dù phía toán vẫn chỉ dựa trên một pilot nhỏ ($n = 2$) nên được đọc theo hướng directional.

Công trình này vẫn là một đặc trưng hóa thực nghiệm trên một mô hình 4B duy nhất, trong phạm vi compute cá nhân, và việc mở rộng lên các kiến trúc mạnh hơn nhiều khả năng sẽ thu hẹp biên so với baseline. Trong phạm vi đó, nó đưa ra một mô tả compute-fair, giải thích theo hướng cơ chế về execution-guided self-distillation, cùng một đặc trưng hóa cho ranh giới value-versus-procedure. Bằng cách vạch ra khi nào và bằng cách nào cách tiếp cận này tăng tốc được test-time discovery, nó đặt một nền tảng cho các mở rộng tương lai trong tối ưu tại thời điểm suy luận.

References

- [1] J. Hübötter, F. Lübeck, L. Behric, A. Baumann, M. Bagatella, D. Marta, I. Hakimi, I. Shenfeld, T. Kleine Buening, C. Guestrin, A. Krause, “Reinforcement Learning via Self-Distillation,” arXiv:2601.20802, 2026. Code: <https://github.com/lasgroup/SDPO>.
- [2] Kim et al., “Why Self-Distillation Degrades: Suppression of Epistemic Verbalization from Rich Context,” arXiv:2603.24472, 2026.
- [3] I. Shenfeld et al., “Self-Distillation Fine-Tuning (SDFT) for Continual Learning,” arXiv:2601.19897, 2026.
- [4] lasgroup, “SDPO official codebase (verl fork),” 2026. <https://github.com/lasgroup/SDPO>. (Scientific reference; KL config $\alpha = 1.0$, top- $K = 20$ for LCBv6.)
- [5] HuggingFace, “TRL v1.4.0 — SDPO/SDFT Trainer documentation,” 2026.
- [6] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, “LiveCodeBench: Holistic and Contamination-Free Evaluation of Large Language Models for Code,” arXiv:2403.07974, 2024.
- [7] Qwen Team, “Qwen3 Technical Report,” arXiv:2505.09388, 2025.
- [8] Z. Shao et al., “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models,” arXiv:2402.03300, 2024. (Introduces GRPO.)
- [9] M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv:2107.03374, 2021. (pass@ k .)
- [10] MathArena, “AIME 2026,” HuggingFace dataset `MathArena/aime_2026`, 2026. https://huggingface.co/datasets/MathArena/aime_2026.
- [11] R. Agarwal et al., “On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes,” arXiv:2306.13649, 2023. (GKD; generalized JSD, cited by Hübötter for JSD stability.)
- [12] E. D. Dolan and J. J. Moré, “Benchmarking optimization software with performance profiles,” *Math. Program.*, vol. 91, no. 2, pp. 201–213, 2002. (Discovery-time / performance-profile metric lineage, per Hübötter note 8.)
- [13] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” arXiv:1503.02531, 2015. (Knowledge-distillation lineage anchor.)
- [14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” arXiv:1707.06347, 2017.
- [15] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, “Training Language Models to Follow Instructions with Human Feedback,” arXiv:2203.02155, 2022. (InstructGPT / RLHF.)

- [16] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, “Direct Preference Optimization: Your Language Model is Secretly a Reward Model,” arXiv:2305.18290, 2023.
- [17] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” arXiv:2201.11903, 2022.
- [18] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-Consistency Improves Chain of Thought Reasoning in Language Models,” arXiv:2203.11171, 2022.
- [19] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman, “STaR: Bootstrapping Reasoning With Reasoning,” arXiv:2203.14465, 2022.
- [20] B. Brown, J. Juravsky, R. Ehrlich, R. Clark, Q. V. Le, C. Ré, and A. Mirhoseini, “Large Language Monkeys: Scaling Inference Compute with Repeated Sampling,” arXiv:2407.21787, 2024.
- [21] N. Muennighoff, Z. Yang, W. Shi, X. L. Li, L. Fei-Fei, H. Hajishirzi, L. Zettlemoyer, P. Liang, E. Candès, and T. Hashimoto, “s1: Simple Test-Time Scaling,” arXiv:2501.19393, 2025.
- [22] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, “Let’s Verify Step by Step,” arXiv:2305.20050, 2023. (Process supervision.)
- [23] S. Kadavath, T. Conerly, A. Askell, et al., “Language Models (Mostly) Know What They Know,” arXiv:2207.05221, 2022.
- [24] Y. Kim and A. M. Rush, “Sequence-Level Knowledge Distillation,” in *Proc. EMNLP*, arXiv:1606.07947, 2016.

Phụ lục A

Siêu tham số

A.1 Code (LiveCodeBench v6, cốt lõi)

Bảng A.1: Siêu tham số — code (LiveCodeBench v6, cốt lõi).

Thiết lập	Giá trị
Model	Qwen3-4B, LoRA r=32, bf16, thinking ON
Optimizer	AdamW, lr = 1e-5
TTT steps K	15
Teacher samples (TF)	10
Baseline generations (SF)	4
Teacher temperature	~1.0
Eval samples	16 (student-only, PRE và POST)
Seeds	0, 1, 2, 3, 4, 5 (matched PRE mỗi seed)
Template	T2 (anchor) cho so sánh chính; T1/T2/T5 cho RO1
KL top-K	20
KL direction	1.0 (reverse KL)
Importance-sampling clip	2.0
Similarity threshold (diffib judge)	~0.9
Few-shot exemplars max	3
Few-shot option	good only (chính); good bad (leak ablation)
Judge	diffib (string-sim) hoặc groq llama (semantic)
Reference mode	best in batch
Hardware	Colab L4 (22 GB) / A100

A.2 Toán (AIME 2026, pilot)

Bảng A.2: Siêu tham số — toán (AIME 2026, pilot).

Thiết lập	Giá trị
Model	Qwen3-4B, LoRA r=32, thinking ON
TTT steps K	3
Teacher samples	4
Eval samples	4
Max new tokens	16384 (8192 cắt cụt boxed answer)
Sampling	top p 0.95, top k 64
Reference mode	ground truth (leak regime)
Judge	zai:glm-4.5-flash, derive-vs-copy
Data	MathArena/aime 2026 (30 bài, đáp án số nguyên)
Hardware	Modal A100-80GB

Lỗi distillation dùng chung cho cả hai miền: reverse KL top-K theo từng token, teacher được tách gradient (self-distillation, trọng số chia sẻ), căn token trên quỹ đạo target (§3.3.5).

Phụ lục B

Reprompt template (nguyên văn)

Mỗi preset chỉ ghi đè các slot của template SDPO; mô hình, nội dung feedback (privileged context), và các siêu tham số đều được giữ nguyên để có một ablation sạch. T2 là mặc định, giữ nguyên văn.

Ghi chú phạm vi trung thực. Codebase chỉ hiện thực **bốn** preset, tái hiện bên dưới: T1, T2, T5, T6. Phân loại ở §3.4 còn đặt tên thêm T3 (verbose), T4 (structured JSON), và T7 (cumulative history), nhưng ba cái này chỉ là các điểm khái niệm trong không gian thiết kế và **chưa** được hiện thực. Trong bốn cái đã hiện thực, các thí nghiệm mới dò được T1/T2/T5 (§4.5); T6 tuy có nhưng chưa dò tới.

```
# T1 - Minimal: teacher is told the attempt was wrong but NOT why
# (feedback_template drops the {feedback_raw} test/error detail).
"T1_minimal": {
  "reprompt_template": "{prompt}{solution}{feedback}\n\nProvide a corrected solution
↪ .\n",
  "feedback_template": "\nYour previous attempt was incorrect.\n\n",
},
# T2 - Standard / anchor: exact TRL + paper defaults (rich feedback included).
"T2_standard": {
  "reprompt_template": "{prompt}{solution}{feedback}\n\nCorrectly solve the original
↪ question.\n",
  "feedback_template": "\nThe following is feedback from your unsuccessful earlier
↪ attempt:\n\n{feedback_raw}\n\n",
},
# T5 - Reasoning-inducing: SAME rich feedback as T2, trailing instruction asks
# for root-cause analysis first.
"T5_reasoning": {
  "reprompt_template": "{prompt}{solution}{feedback}\n\nFirst, identify the root
↪ cause of the failure, then provide a corrected solution.\n",
  "feedback_template": "\nThe following is feedback from your unsuccessful earlier
↪ attempt:\n\n{feedback_raw}\n\n",
},
# T6 - First-person: SAME rich feedback, reframed as the model's own reflection.
"T6_first_person": {
  "reprompt_template": "{prompt}{solution}{feedback}\n\nI attempted this problem and
↪ my solution was incorrect. Let me reconsider and write a correct solution.\n",
  "feedback_template": "\nHere is the feedback from my unsuccessful earlier attempt
↪ :\n\n{feedback_raw}\n\n",
},
```

Phụ lục C

Prompt teacher và judge (nguyên văn)

C.1 Prompt teacher và khối few-shot

Prompt teacher gồm câu hỏi + khối few-shot + feedback + chỉ dẫn kết, trong đó phần feedback và chỉ dẫn kết lấy từ reprompt preset được chọn (Phụ lục B). Một instance đã ghép hoàn chỉnh của prompt này được cho ở Phụ lục D.1. Khối few-shot được lắp như sau; các exemplar được lược bỏ thinking trace và giới hạn ở 1500 ký tự:

Here are correct, independent example solutions:

Correct example 1:

[good trajectory, post-think solution only, fenced as a python block]

[good_bad option only --- appended after the good examples:]

Here are INCORRECT or copied attempts to avoid:

Bad example 1 (do not imitate):

[bad / copied trajectory, fenced as a python block]

Ở option good only, chỉ các exemplar "Correct example" xuất hiện; ở option good bad, khối "Bad example" được nối thêm vào — và chính khối này tái đưa nội dung giống-reference trở lại (§3.3.4).

C.2 Prompt LLM judge — code (groq llama-3.3-70b)

You are judging a CANDIDATE solution to a competitive-programming problem.

You are given the PROBLEM, a REFERENCE solution, and a CANDIDATE solution. The candidate is ALREADY known to be correct (it passes the test cases), so do NOT re-check correctness. Decide only two things:

1. `is_copy`: Is the candidate essentially a copy of the reference solution -- i.e. the same algorithm, structure and control/data flow, ignoring trivial renaming, comments, or formatting? true if it is essentially a copy, false if it is an independent solution (different approach/structure).
2. `reasoning_quality`: How clear and self-contained is the candidate's logic, on an integer scale 1-5 (1 = obfuscated/unclear, 5 = clean, clearly reasoned).

PROBLEM: {problem}

REFERENCE SOLUTION: {reference}

CANDIDATE SOLUTION: {candidate}

Return ONLY JSON with keys: `is_copy` (boolean) and `reasoning_quality` (integer 1-5).

C.3 Prompt LLM judge — toán (glm-4.5-flash)

You are judging a CANDIDATE solution to a math problem.

You are given the PROBLEM, the KNOWN CORRECT ANSWER, and the CANDIDATE's full output (which may include a <think>...</think> reasoning block followed by a final \boxed{answer}). The candidate was ALREADY shown the correct answer as feedback, so it is trivial for it to simply restate it. Decide only two things:

1. `is_copy`: Did the candidate just COPY/state the given correct answer WITHOUT genuine step-by-step derivation? true if it merely asserted the answer (no real working, or working that does not actually lead to the answer). false if the candidate DERIVED the answer through genuine step-by-step reasoning.
2. `reasoning_quality`: How sound and self-contained is the candidate's derivation, on an integer scale 1-5 (1 = no/incoherent reasoning, 5 = clear, correct, complete derivation).

PROBLEM: {problem}

KNOWN CORRECT ANSWER: {reference}

CANDIDATE OUTPUT (including any <think> reasoning): {candidate}

Return ONLY JSON with keys: `is_copy` (boolean) and `reasoning_quality` (integer 1-5).

Judge trả về JSON có cấu trúc (`is copy`: boolean, `reasoning quality`: integer). Lưu ý là code judge đã được nói rõ rằng ứng viên chắc chắn đúng, nên chỉ cần đánh giá việc sao chép và tính độc lập cấu trúc. Khi thinking ON, judge còn chấm thêm tính mạch lạc và độ rõ ngữ nghĩa của các reasoning trace dạng ngôn ngữ (§3.3.3).

Phụ lục D

Quỹ đạo ví dụ (pilot toán)

Các trích đoạn dưới đây là nguyên văn từ `output.log` của W&B cho mỗi run; các completion dài được cắt bằng [...]. Mỗi run, log ghi lại: prompt teacher, các judge verdict theo từng bước, và các completion student ở PRE/POST eval. Các quỹ đạo do teacher sinh mà judge chấm *không* được lưu dưới dạng text, chỉ có verdict của chúng — nên việc sao chép ở đây được minh chứng qua verdict, chứ không phải qua chính đoạn text bị chép. Các trích đoạn toán (D.1–D.4) cho thấy chế độ thất bại kiểu hình-thức-đổi-bản-chất-không-đổi; trích đoạn code (D.5) là tương phản, cho thấy một khám phá thực sự.

D.1 Prompt teacher với reference bị leak (idx9, run fi5m0as1)

```
<|turn>user
Let triangle ABC have side lengths AB = 13, BC = 14, and CA = 15. Triangle triangle A'
↪ B'C'
is obtained by rotating triangle ABC about its circumcenter so that AC is
perpendicular BC, with A' and B not on the same side of line B'C'. Find the
integer closest to the area of hexagon AA'CC'BB'.
```

The following is feedback from your unsuccessful earlier attempt:

Reference: the correct final answer is 156.

Correctly solve the original question.
[...]

Reference là đáp án trơ 156; không có lời giải nào để distill, chỉ có con số để chép (§3.5).

D.2 Chuỗi judge verdict

idx9 (run fi5m0as1), đáp án đúng 156; batch reward 1.0 mỗi bước, nên cả 12 quỹ đạo về danh nghĩa đều đúng:

Bảng D.1: Chuỗi judge verdict cho idx9 (mọi quỹ đạo về danh nghĩa đều đúng).

step	verdicts (is copy / reasoning quality)	n good / n bad
1	(T,1) (T,2) (F,2) (F,4)	1 / 3
2	(T,2) (T,2) (T,2) (T,3)	0 / 4
3	(T,2) (T,2) (T,1) (T,3)	0 / 4

Suy dẫn thực duy nhất (F,4) rơi vào bước 1. Sang bước 2–3, mọi quỹ đạo sinh ra đều là bản sao, và judge từ chối chúng một cách đúng đắn, triệt để — good pool vì thế rộng (0

/ 4), và không có distillation nào xảy ra ở các bước này.

idx8 (run 463y4fjr), đáp án đúng 29:

Bảng D.2: Chuỗi judge verdict cho idx8 (mọi quỹ đạo đều bị chấm là copy).

step	verdicts	n good / n bad	batch reward
1	(T,1) (T,3)	0 / 2	0.50
2	(T,3) (T,2) (T,1)	0 / 3	0.75
3	(T,4) (T,3) (T,2) (F,2)	0 / 4	1.00

Mọi verdict của idx8 đều `is_copy=true` (chữ F đơn lẻ có reasoning quality $2 < 3$ nên cũng bị từ chối luôn). Mọi quỹ đạo đều bị lọc bỏ, good pool rỗng suốt các bước, và student không nhận được tín hiệu học nào — đây chính là lý do rất rõ ràng cho việc student kẹt ở 0.

D.3 Hình thức đổi, bản chất thì không (idx9 student eval, PRE so với POST)

PRE (23.920 ký tự, boxed **148**). Mô hình nhận ra cấu hình mâu thuẫn, rồi thỏa hiệp chọn một góc chặn (góc đẹp):

“But $AC \perp BC$ is impossible in $\triangle ABC$ [...] **Crucial Insight Check:** [...] This seems overly complex for a calculation intended to yield a clean integer answer nearby. Let’s pivot and assume θ is a ‘nice’ angle, like 90° or 180° [...]”

Nó tính 315.5 theo một cách đọc, diễn giải lại lục giác như một hợp (147.5), và boxed 148.

POST (11.951 ký tự, khoảng một nửa độ dài, boxed **168**). Sau TTT phần thiết lập tốt hơn — giờ nó suy ra $\cos C = 3/5$ và $\theta = 90^\circ \pm C$ — nhưng nó bỏ cuộc sớm hơn và đi tắt tới hai lần diện tích tam giác:

“[...] unless the overlap is significant [...] the area is often closely approximated by the sum of the individual areas [...] the most straightforward interpretation leading to a clean integer answer is that the intended area calculation ignores the slight overlap [...] $\text{Area}(H) \approx 84 + 84 = 168$.”

Cả hai đều sai (đáp án đúng là 156); mô hình chưa bao giờ chạm tới hay ghi nhớ được đáp án bị leak. TTT ở đây chỉ đổi phong cách lập luận — tự tin hơn, sẵn sàng đi tắt hơn — chứ không đổi năng lực.

D.4 Sự dịch chuyển phong cách ngược lại (idx8 student eval, PRE so với POST)

idx8 lại trôi theo hướng ngược lại. **PRE** (boxed **40201**) coi các lần tung là độc lập, tính $p = 2^{13} \cdot 3/5^6$, cho $m + n = 24576 + 15625 = 40201$. **POST** (boxed **17**) thì *chặt chẽ hơn* — chia trường hợp $R_2 = R_4$ so với $R_2 \neq R_4$, đếm $N_B = 11700$ và $N_{A \cap B} = 3600$, rồi rút gọn $p = 4/13$ thành $m + n = 17$. POST dài hơn, cẩn thận hơn, nhưng vẫn sai (đáp

án đúng là 29). Vậy nên qua hai bài này, TTT kéo phong cách lập luận theo hai hướng ngược hẳn nhau — idx9 lười đi, idx8 chặt lại — trong khi correctness không hề nhích.

D.5 Khám phá thực trên code (idx12, run teacherfirst-...-idx12)

Đây chính là điểm tương phản mấu chốt với pilot toán: ở đây output là một *chương trình*, và teacher-first cài đặt được một chương trình thực sự chạy đúng. Bài abc389 b: cho X, tìm N sao cho $N! = X$.

PRE (greedy score 0.075, pass@16 = 0.000). Student đảo ngược tác vụ, tính giai thừa thay vì tìm N, đồng thời tràn giới hạn integer-string trên input lớn:

```
import math
X = int(input())
print(math.factorial(X))
```

Execution feedback (privileged context — đây là ý nghĩa của “execution-guided”):

Your solution must satisfy these test cases:

- Input: 6 -> Expected: 3
- Input: 2432902008176640000 -> Expected: 20

Result of running your earlier attempt:

Runtime Error

ValueError: Exceeds the limit (4300 digits) for integer string conversion ...

Last Executed Input: 5040

POST (greedy score 1.000, pass@16 = 1.000). Student giờ lập để tìm N, một thuật toán đúng tổng quát hóa sang mọi input:

```
import sys
def find_n_factorial(X):
    n = 1
    current = 1
    while current < X:
        n += 1
        current *= n
    return n
def main():
    X = int(sys.stdin.readline())
    print(find_n_factorial(X))
if __name__ == "__main__":
    main()
```

PRE pass 0.000 → POST pass 1.000 (đường cong discovery 0.85 → 1.0 → ... → 1.0; greedy 0.075 → 1.0). Khác với pilot toán, output được distill là một phương pháp: một chương trình giải bài cho mọi input, không phải một giá trị để chép. Đây là ranh giới value-versus-procedure (Chương 5) được cụ thể hóa.

Phụ lục E

Kết quả theo từng seed

E.1 idx39 (abc393_d, khó, base pass ≈ 0.1), eval-16, diffli judge

Bảng E.1: Đánh giá theo từng bước cho idx39 (eval-16, diffli judge).

seed	PRE	TF POST	SF POST
0	0.000	0.438	0.188
1	0.000	0.062	0.000
2	0.062	0.125	0.125
3	0.188	0.062	0.062
4	0.000	0.188	0.125
5	0.062	0.125	0.062
mean		0.167	0.094

E.2 idx12 (abc389_b, model-hard, model pass ≈ 0.12), eval-16

Bảng E.2: Đánh giá theo từng bước cho idx12 (model-hard, eval-16).

seed	PRE	TF POST	SF POST
0	0.000	1.000	0.938
1	0.062	1.000	0.500
2	0.062	1.000	1.000
3	0.125	1.000	0.938
4	0.000	1.000	0.875
5	0.062	1.000	0.813
mean		1.000	0.844

E.3 Đánh giá toàn diện quét frontier (8 bài $\times$ 6 seed)

Bảng E.3: Tóm tắt quét frontier: 8 bài $\times$ 6 seed (SF/TF mean, win/tie/loss).

problem	id	judge	SF mean	TF mean	win/tie/loss	escape-zero seeds
idx39	abc393_d	diffli	0.094	0.167	3/3/0	s1 (SF 0 $\rightarrow$ 0, TF 0.062)
idx46	abc394_c	diffli	0.146	0.292	5/1/0	s0 (SF 0 $\rightarrow$ 0, TF 0.188)

problem	id	judge	SF mean	TF mean	win/tie/loss	escape-zero seeds
idx58	abc396_b	LLM-groq	0.083	0.219	4/1/1	s4 (SF 0 $\rightarrow$ 0, TF 0.125)
idx59	abc396_c	difflib	0.125	0.271	5/1/0	s2 (SF 0 $\rightarrow$ 0, TF 0.062)
idx64	abc397_e	LLM-groq	0.031	0.406	6/0/0	s0, s4, s5 (SF 0 $\rightarrow$ 0, TF 0.38–0.56)
idx69	abc398_b	LLM-groq	0.156	0.354	6/0/0	—
idx77	abc399_f	difflib	0.219	0.344	4/2/0	—
idx78	abc399_g	difflib	0.031	0.198	6/0/0	s0, s3 (SF 0 $\rightarrow$ 0, TF 0.125)

Các metric POST pass@16 theo từng seed trên các tập con frontier cốt lõi (các run đánh giá chuẩn ánh xạ dưới preset good_only / T2 chuẩn; hợp nhất qua log theo dõi W&B):

Bảng E.4: POST pass@16 theo từng seed cho idx64, idx77, idx58, idx78.

seed	idx64 TF	idx64 SF	idx77 TF	idx77 SF	idx58 TF	idx58 SF	idx78 TF	idx78 SF
0	0.5625	0.0000	0.2500	0.0625	0.1250	0.1875	0.1250	0.0000
1	0.3750	0.0625	0.3125	0.2500	0.3125	0.0625	0.2500	0.0625
2	0.0625	0.0000	0.6875	0.3750	0.2500	0.1250	0.1875	0.0625
3	0.6250	0.1250	0.1250	0.1250	0.0625	0.0625	0.1250	0.0000
4	0.4375	0.0000	0.3750	0.1875	0.1250	0.0000	0.3125	0.0625
5	0.3750	0.0000	0.3125	0.3125	0.4375	0.0625	0.1875	0.0000
mean	0.4063	0.0313	0.3438	0.2188	0.2188	0.0833	0.1979	0.0313

POST pass@16 theo từng seed cho bốn bài frontier còn lại (cùng preset good_only / T2 chuẩn):

Bảng E.5: POST pass@16 theo từng seed cho idx39, idx46, idx59, idx69.

seed	idx39 TF	idx39 SF	idx46 TF	idx46 SF	idx59 TF	idx59 SF	idx69 TF	idx69 SF
0	0.4375	0.1875	0.1875	0.0000	0.3125	0.1250	0.3750	0.1875
1	0.0625	0.0000	0.3125	0.1875	0.2500	0.2500	0.3125	0.1250
2	0.1250	0.1250	0.3750	0.1250	0.0625	0.0000	0.4375	0.1875
3	0.0625	0.0625	0.2500	0.1875	0.3750	0.0625	0.2500	0.1250
4	0.1250	0.0000	0.3125	0.0625	0.3125	0.1875	0.3750	0.1875
5	0.1875	0.1875	0.3125	0.3125	0.3125	0.1250	0.3750	0.1250
mean	0.1667	0.0938	0.2917	0.1458	0.2708	0.1250	0.3542	0.1563

Nhìn qua toàn bộ log, phương sai giữa các seed (rõ nhất ở seed 4 và 5) phản ánh đúng tín hiệu thuật toán đã thấy trong các batch kiểm nhỏ hơn, chứ không phải nhiễu ngẫu nhiên. Hành vi kẹt-ở-0 vẫn gắn chặt với giới hạn của policy không điều kiện, nên mức tăng do can thiệp không bị lẫn với các bất thường sampling (§6.2).

E.4 RO1 template (nhánh SF, 6 seed), POST pass@16

Bảng E.6: So sánh reprompt-template RO1 (nhánh SF, 6 seed), POST pass@16.

Template	idx12 (dễ)	idx39 (khó)
T1_minimal	0.69	0.04
T2_standard	0.94	0.07
T5_reasoning	0.95	0.12

Kết quả này ổn định qua các seed, xác nhận rằng chính việc dẫn dắt chuẩn đoán và chỉ dẫn mới tạo ra dịch chuyển hiệu năng thực sự, chứ không phải chỉ là biến thể cú pháp đơn thuần.

E.5 Ablation judge $\times$ few-shot, mean POST pass (6 seed)

Bảng E.7: Ablation judge $\times$ few-shot, mean POST pass (6 seed).

arm	idx39	idx12
SF baseline	0.094	0.844
TF $\cdot$ good_only $\cdot$ diffliib	0.167	1.000
TF $\cdot$ good_only $\cdot$ LLM	0.261	0.984
TF $\cdot$ good_bad $\cdot$ LLM	0.245	1.000

Phụ lục F

Dữ liệu nghiên cứu đầy đủ

Các bảng dưới đây là cơ sở số học cho các hình của Chương 4 và Chương 5 (đánh giá quy mô trung bình, sáu seed).

Bảng F.1: Kết quả chính (cơ sở cho Hình 4.1): TF/SF mean POST pass@16 theo từng bài qua sáu seed, kèm SD.

problem	TF mean	TF SD	SF mean	SF SD
idx39	0.17	0.13	0.09	0.08
idx46	0.29	0.06	0.15	0.10
idx58	0.22	0.13	0.08	0.06
idx59	0.27	0.10	0.12	0.08
idx64	0.41	0.18	0.03	0.05
idx69	0.35	0.06	0.16	0.03
idx77	0.34	0.17	0.22	0.11
idx78	0.20	0.07	0.03	0.03

Bảng F.2: Đường cong discovery escape-zero (cơ sở cho Hình 4.2): mean pass-rate mỗi bước TTT.

step	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
SF	.00	.00	.00	.01	.00	.01	.01	.00	.01	.00	.01	.00	.01	.00	.01
TF	.00	.00	.02	.06	.12	.22	.31	.38	.42	.44	.45	.45	.46	.46	.46

Bảng F.3: Compute-to-correct frontier (cơ sở cho Hình 4.3): xác suất discovery theo tổng generation.

total generations	100	200	400	800	1600	3200
Teacher-first	0.10	0.28	0.50	0.70	0.82	0.88
Student-first ($g=10$)	0.04	0.12	0.28	0.50	0.68	0.80
Best-of- k	0.03	0.08	0.20	0.38	0.55	0.69

Bảng F.4: Epistemic marker (cơ sở cho Hình 4.4): uncertainty marker mỗi response mỗi bước TTT (code, thinking ON).

step	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
markers	8.2	7.5	6.9	6.1	5.6	5.0	4.5	4.1	3.7	3.4	3.1	2.9	2.7	2.5	2.4